DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT - I : Introduction & Entity Relationship Model

Easy handwritten notes with diagrams for students

Topics in this unit

1. Database system, characteristics (Database vs File system)
2. Database users, advantages and applications of DBMS
3. Data models, schema, instance and data independence
4. Three-schema architecture, DBMS structure, environment
5. Centralized and Client-Server architecture
6. ER model: entities, attributes, relationships, constraints
7. Sub classes, super class, inheritance, specialization, generalization

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Introduction: Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database. Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Part 1 : Introduction to Database Systems

1.1 What is a Database System?

Definition: **Data** = raw facts (e.g. 101, Ravi, 20). **Information** = processed data with meaning. A **database** is an organised collection of related data stored so that it can be easily accessed, managed and updated. A **DBMS** (Database Management System) is the software that creates, stores, retrieves and controls the database. Database + DBMS + Application programs = **Database System**.

- Goal 1: **Store** large data safely and permanently.
- Goal 2: **Retrieve** it quickly using queries (e.g. SQL).
- Goal 3: **Share** data among many users with security and correctness.
- Examples of DBMS: Oracle, MySQL, PostgreSQL, SQL Server, MongoDB, IBM DB2.

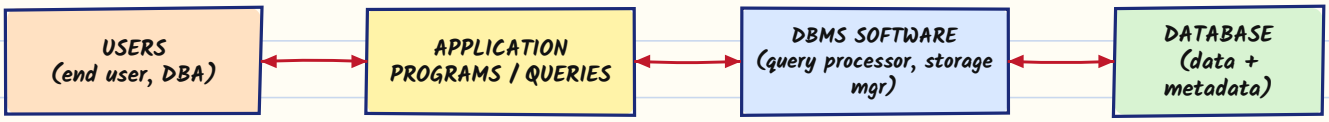


Fig 1.1 : Components of a database system

* **Remember:** Metadata = 'data about data' (table names, column types, constraints). It is stored in the **system catalog / data dictionary**.

1.2 File System vs Database System (Characteristics)

Before DBMS, data was kept in separate files (one per application). This caused many problems, which a DBMS solves.

Point	File system	Database system (DBMS)
Redundancy	Same data repeated in many files	Controlled - data stored once
Consistency	Copies may disagree (inconsistency)	Single copy keeps data consistent
Data access	New program needed for each query	Query language (SQL) gives ad-hoc access
Data isolation	Data scattered in different formats	Common structure and format
Integrity	Rules coded inside every program	Constraints declared once in the DBMS
Concurrency	Simultaneous updates cause errors	Concurrency control handles many users
Security	Weak, OS-level only	User accounts, privileges, views
Recovery	Hard to restore after crash	Backup + log-based recovery
Atomicity	Half-done updates remain	Transaction is all-or-nothing

Main characteristics of the database approach

- **Self-describing** - catalog stores the structure of the database.
- **Insulation** between programs and data (program-data independence).
- **Multiple views** - each user sees only the part he needs.
- **Data sharing** and multi-user transaction processing.

- **Controlled redundancy**, integrity constraints, security and backup/recovery.

1.3 Advantages and Disadvantages of DBMS

Advantages	Disadvantages
Reduces redundancy and inconsistency	High initial cost (software, hardware, training)
Data sharing among users and applications	Complexity - needs skilled DBA
Enforces integrity and security	Performance overhead for very small applications
Backup and recovery, concurrency control	Single point of failure if not replicated
Data independence, faster development	Upgrades / conversion can be costly

1.4 Database Users

User type	Role
Database Administrator (DBA)	Controls the whole DB: schema definition, storage, security, backup, performance tuning, grants permissions.
Database Designer	Decides what data to store and designs the structure (ER diagram, tables).
Application Programmer	Writes programs (Java, Python, PHP) that access the DB using SQL.
Sophisticated user	Analysts / scientists who write their own complex queries directly.
Naive (end) user	Uses ready-made forms/apps, e.g. bank clerk, ticket booking user.
Casual user	Occasional user, e.g. manager who sometimes queries the DB.

1.5 Database Applications

Area	Use of database
Banking	Accounts, transactions, loans, ATM
Railways / Airlines	Reservation, schedules, tickets
Universities	Student records, results, attendance, library
E-commerce	Products, orders, payments, customers (Amazon, Flipkart)
Telecom / Social media	Call records, bills, user profiles, posts
Healthcare / Government	Patient records, Aadhaar, tax, voter lists

Part 2 : Data Models, Schema and Architecture

2.1 Data Models (brief introduction)

A **data model** is a collection of concepts used to describe the structure of a database (data, relationships, constraints). It hides storage details.

Data model	Simple explanation
Hierarchical	Data in a tree - one parent, many children (IBM IMS). Fast but rigid.
Network	Graph - a record can have many parents (CODASYL). Flexible but complex.
Relational	Data in tables (rows and columns) linked by keys. Most popular (MySQL, Oracle).
Entity-Relationship (ER)	High-level conceptual model using entities and relationships; used for design.

Data model	Simple explanation
Object-oriented	Stores data as objects with attributes and methods (db4o).
Object-relational	Relational model + object features (PostgreSQL, Oracle).
NoSQL / Semi-structured	Document (MongoDB, JSON), key-value (Redis), column, graph (Neo4j) - for big, flexible data.

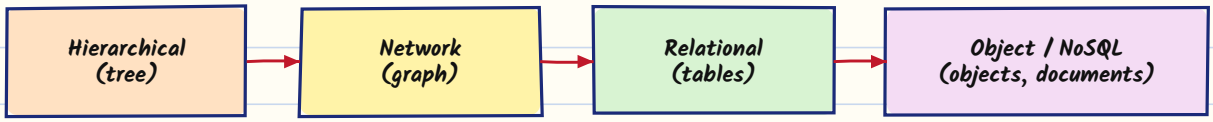


Fig 1.2 : Evolution of data models

2.2 Schema and Instance

Term	Meaning	Changes?
Schema	The overall design / structure of the database (table names, columns, types, constraints). Like the blueprint of a building.	Rarely
Instance	The actual data stored in the database at a particular moment (snapshot). Like the people living in the building.	Very often

Example schema: **STUDENT(Roll_no, Name, Age, Dept)**. Example instance: **(101, Ravi, 20, CSE), (102, Sita, 19, ECE)**.

Types of schema: **Physical schema** (storage), **Logical (conceptual) schema** (tables, relations) and **External schema / subschemas** (user views).

2.3 Three-Schema (Three-Tier) Architecture

Proposed by ANSI-SPARC. It separates the database into 3 levels so that users are isolated from the physical details.

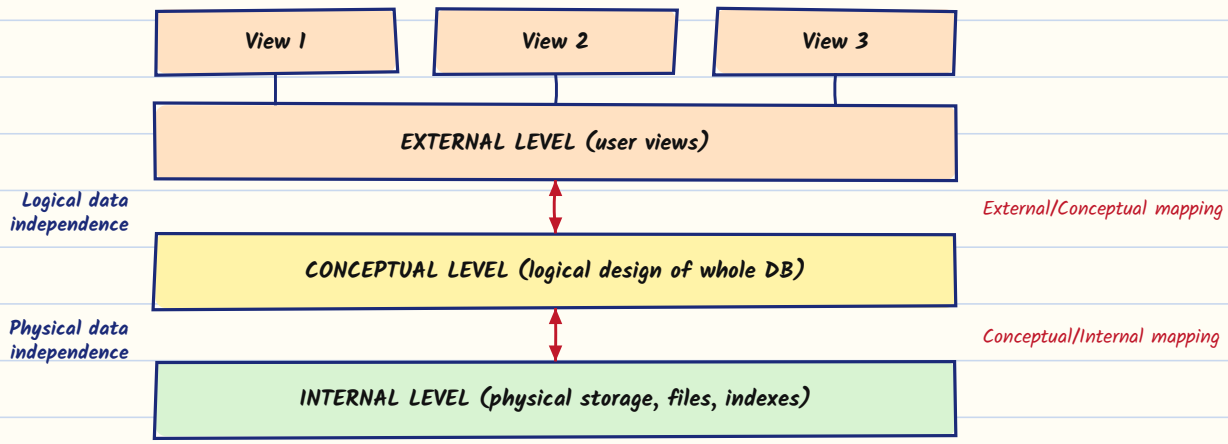


Fig 1.3 : Three-schema architecture and data independence

Level	Describes	Who uses it
External (view) level	Part of DB relevant to a user group; hides the rest	End users, application programmers
Conceptual (logical) level	Entities, attributes, relationships, constraints of the whole DB	DBA, designers

Level	Describes	Who uses it
Internal (physical) level	How data is stored: file organisation, indexes, access paths	DBMS / system programmers

2.4 Data Independence

Data independence = the ability to change the schema at one level without having to change the schema at the next higher level. It is the main benefit of the three-schema architecture.

Type	Meaning	Example
Physical data independence	Change internal (physical) schema without changing conceptual schema	Add an index, move file to a new disk, change file organisation
Logical data independence	Change conceptual schema without changing external views / programs	Add a new column or table; programs using old views still work

* **Remember:** Logical independence is **harder** to achieve than physical independence, because application programs depend strongly on the logical structure.

2.5 Database System Structure (Components of DBMS)

A DBMS has two main parts: the **Query Processor** and the **Storage Manager**, working on top of the disk storage.

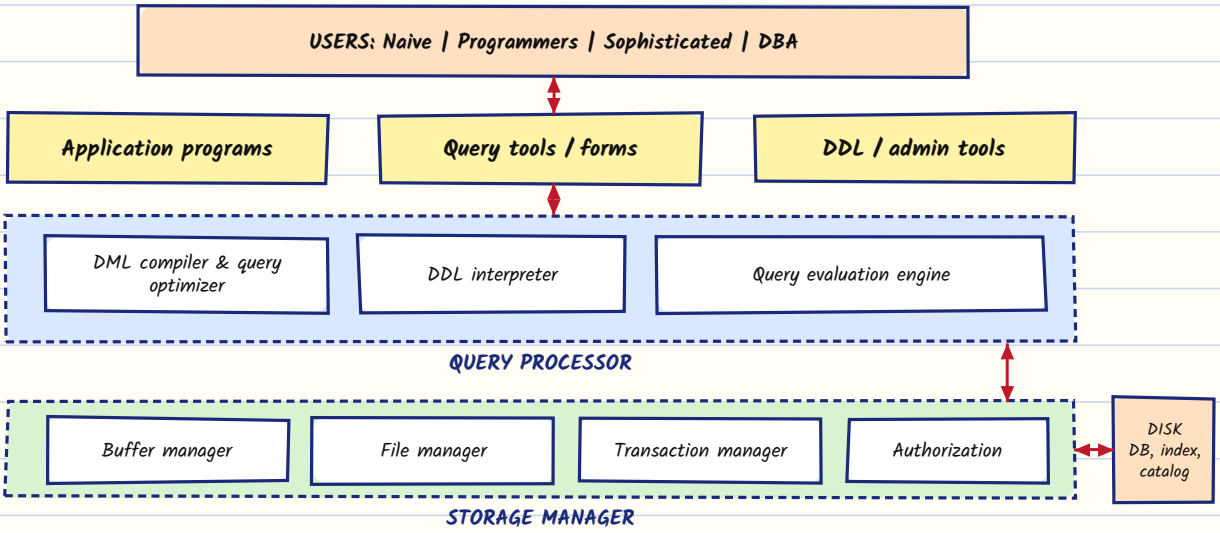


Fig 1.4 : Structure of a DBMS

Component	Job
DDL interpreter	Reads schema definitions and stores them in the catalog.
DML compiler / optimizer	Converts a query into an efficient low-level plan.
Query evaluation engine	Executes the plan and returns the result.
Buffer manager	Moves data between disk and main memory.
File manager	Manages space on disk and the data structures used.
Transaction manager	Ensures ACID properties - concurrency and recovery.
Authorization & integrity manager	Checks access rights and integrity constraints.

2.6 Database Environment and Languages

- **DDL** (Data Definition Language) - CREATE, ALTER, DROP: defines the schema.
- **DML** (Data Manipulation Language) - SELECT, INSERT, UPDATE, DELETE: handles data.
- **DCL** (Data Control Language) - GRANT, REVOKE: permissions.
- **TCL** (Transaction Control Language) - COMMIT, ROLLBACK, SAVEPOINT.
- Environment = DBMS software + hardware + data + users + procedures (+ network).

2.7 Centralized and Client-Server Architecture

Centralized architecture

DBMS, data and applications all run on **one computer** (mainframe or PC). Users connect through dumb terminals. Simple, but the single machine is a bottleneck and a single point of failure.

Client-Server architecture

Work is split: the **client** runs the user interface and application logic; the **server** runs the DBMS and stores the data. They communicate over a network.

Two-tier

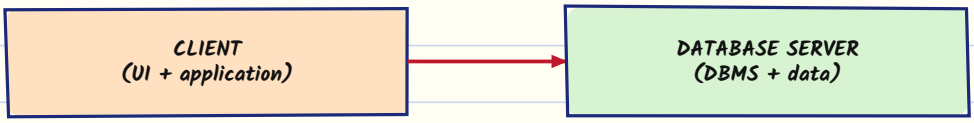


Fig 1.5 : Two-tier architecture (client talks to DB directly via ODBC/JDBC)

Three-tier



Fig 1.6 : Three-tier architecture (used in web applications)

Point	Centralized	Client-Server
Processing	All on one machine	Shared between client and server
Cost / scaling	Cheap but hard to scale	Scalable, add more clients/servers
Failure	Whole system stops	Only affected part stops
Example	Old mainframe banking	Web apps, ERP, online shopping

Part 3 : Entity Relationship (ER) Model

3.1 Introduction

The **ER model** (Peter Chen, 1976) is a high-level **conceptual** data model used to design a database before creating tables. It describes the real world as **entities**, their **attributes** and the **relationships** among them, drawn as an **ER diagram**.

Steps: requirements -> ER diagram -> convert to relational schema -> create tables.

3.2 Entity, Entity Set and Attributes

- **Entity** - a real-world thing that can be identified (a student, a book, a course).
- **Entity set** - collection of entities of the same type (all students). Shown as a rectangle.

- **Strong entity** - has its own key attribute. **Weak entity** - no key of its own; depends on an owner (strong) entity through an identifying relationship (e.g. DEPENDENT of EMPLOYEE).

Types of attributes

Attribute	Meaning	Example
Simple (atomic)	Cannot be divided	Age, Roll_no
Composite	Can be divided into parts	Name = First + Last; Address
Single-valued	One value per entity	DOB
Multi-valued	Many values per entity	Phone numbers, Skills
Derived	Calculated from other attribute	Age from DOB
Key attribute	Uniquely identifies an entity	Roll_no
Null attribute	Value unknown / not applicable	Middle name

ER diagram symbols

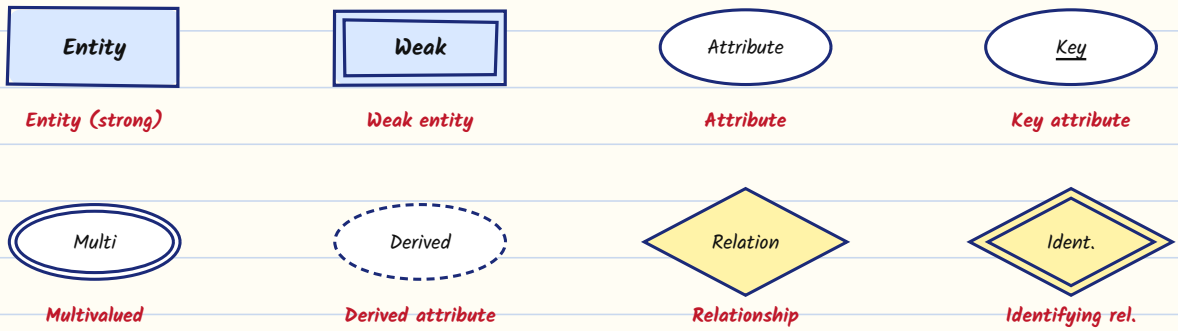


Fig 1.7 : Basic ER diagram notation

3.3 Relationship, Relationship Set and Degree

- **Relationship** - an association among entities (Student ENROLLS in Course).
- **Relationship set** - collection of relationships of the same type. Shown as a diamond.
- **Degree** - number of entity sets taking part: unary (1), binary (2), ternary (3).
- A relationship can have its own attributes (e.g. Grade in ENROLLS).

3.4 Constraints on Relationships

(a) Mapping cardinality (cardinality ratio)

Type	Meaning	Example
One-to-One (1:1)	One A is related to at most one B	Person - Passport
One-to-Many (1:N)	One A relates to many B; each B to one A	Department - Employee
Many-to-One (N:1)	Many A relate to one B	Employee - Department
Many-to-Many (M:N)	Many A relate to many B	Student - Course

(b) Participation constraint

- **Total participation** (double line) - every entity must take part in the relationship (every Employee works for a Department).
- **Partial participation** (single line) - some entities may not take part.

Example ER diagram : STUDENT enrolls in COURSE

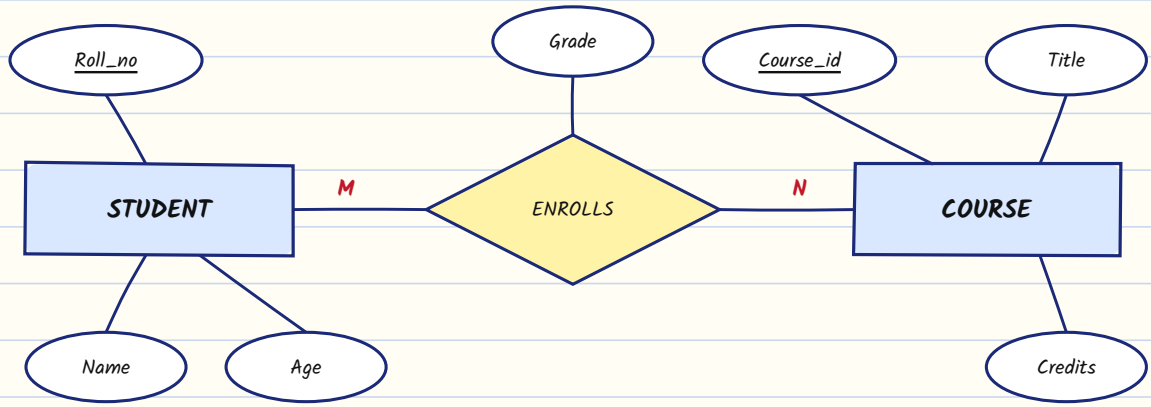


Fig 1.8 : ER diagram with M:N relationship and relationship attribute

3.5 Super Class, Sub Class, Inheritance

- **Super class** - the general entity type (EMPLOYEE).
- **Sub class** - a more specific type that is a subset (ENGINEER, SECRETARY). Relationship is called **IS-A**.
- **Inheritance** - a sub class automatically gets all attributes and relationships of its super class, plus its own extra attributes.

Specialization (top-down)

Start with a general entity and divide it into more specific sub classes based on some difference. EMPLOYEE -> ENGINEER, SECRETARY, TECHNICIAN.

Generalization (bottom-up)

Start with several specific entities and combine their common features into one general super class. CAR and TRUCK -> VEHICLE.

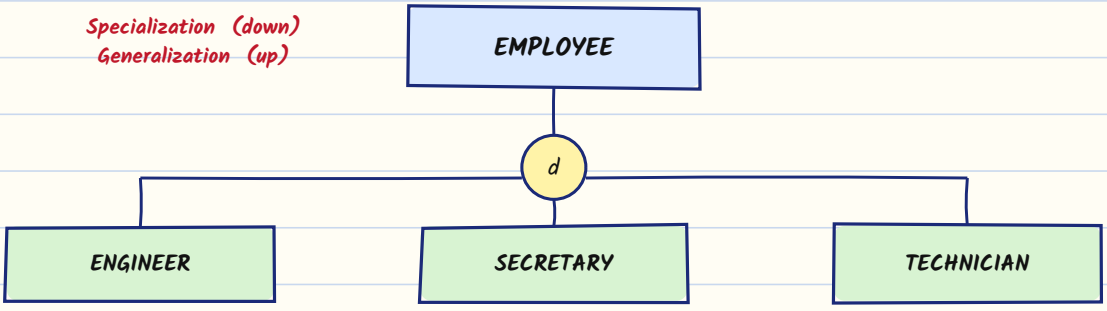


Fig 1.9 : Specialization / generalization (circle 'd' = disjoint, 'o' = overlapping)

Constraints on specialization

Constraint	Meaning
Disjoint (d)	An entity can belong to at most ONE sub class (a person is either Male or Female).
Overlapping (o)	An entity can belong to MORE than one sub class (a person can be both Student and Employee).
Total (double line)	Every super class entity must be in some sub class.
Partial (single line)	Some super class entities may belong to no sub class.

* **Remember:** Specialization = **top-down** (split). Generalization = **bottom-up** (merge). Both are shown with the same diagram; only the direction of thinking differs.

Quick Revision - Unit 1

- DBMS = software to create, store, retrieve and control a database; database = organised related data.
- File system problems: redundancy, inconsistency, isolation, weak security, no concurrency - DBMS solves these.
- Users: DBA, designer, application programmer, sophisticated, naive, casual.
- Data models: hierarchical, network, relational, ER, object-oriented, NoSQL.
- Schema = structure (rarely changes); Instance = data now (changes often).
- Three levels: External -> Conceptual -> Internal. Physical & logical data independence.
- DBMS parts: Query processor + Storage manager. Languages: DDL, DML, DCL, TCL.
- Architecture: centralized, 2-tier and 3-tier client-server.
- ER: entity (rectangle), attribute (ellipse), relationship (diamond); keys underlined.
- Cardinality 1:1, 1:N, M:N; participation total / partial; weak entity.
- Specialization (top-down), generalization (bottom-up), inheritance, disjoint/overlapping.

Possible exam questions

1. Define DBMS. Compare file system and database system.
2. Explain the advantages of DBMS and the different types of database users.
3. Explain the three-schema architecture. How does it give data independence?
4. Differentiate schema and instance; physical and logical data independence.
5. Explain the structure (components) of a DBMS with a neat diagram.
6. Explain centralized and client-server (2-tier, 3-tier) architectures.
7. Explain ER model concepts: entity, attributes (types), relationships, constraints.
8. Explain specialization, generalization and inheritance with ER diagrams.
9. Draw an ER diagram for a University / Library / Hospital database.

DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT – II : Relational Model & Basic SQL

Easy handwritten notes with diagrams for students

Topics in this unit

1. Relational model: domain, attribute, tuple, relation
2. Importance of null values
3. Constraints: domain, key and integrity constraints
4. Relational algebra (operators with examples)
5. Relational calculus (tuple and domain)
6. Basic SQL: schema, data types, CREATE / ALTER
7. DML operations: INSERT, DELETE, UPDATE

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory – that is the fastest way to remember it for exams.

Syllabus (as given)

Relational Model: Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus. BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Part I : The Relational Model

1.1 Introduction

The **relational model** was proposed by **E.F. Codd (1970)**. It stores all data in simple two-dimensional **tables** called relations. It is the basis of almost every modern DBMS (MySQL, Oracle, PostgreSQL, SQL Server). Its strength is simplicity, a strong mathematical base (set theory, logic) and the standard language SQL.

1.2 Basic Terms

Term	Meaning	Table word
Relation	A table with a name; a set of tuples	Table
Tuple	One row = one record of the relation	Row / record
Attribute	A named column; property of the relation	Column / field
Domain	Set of allowed (atomic) values for an attribute	Data type + range
Degree (arity)	Number of attributes (columns)	No. of columns
Cardinality	Number of tuples (rows)	No. of rows
Relation schema	Name + list of attributes: STUDENT(Roll_no, Name, Age, Dept_id)	Table design
Relation instance	The set of tuples present at a given time	Table data

STUDENT (relation)

Attribute (column)

Roll_no	Name	Age	Dept_id
101	Ravi	20	D1
102	Sita	19	D2
103	Arun	21	D1

Tuple (row)

Degree = 4 , Cardinality = 3 , Domain of Age = integers (e.g. 15 to 60)

Fig 2.1 : A relation with its parts

Properties of a relation

- Relation name is unique; attribute names in a relation are unique.
- Every value is **atomic** (single, not a list) - this is first normal form.
- **No duplicate tuples** (a relation is a set).
- **Order of rows and columns does not matter**.

1.3 Importance of NULL Values

NULL is a special marker meaning the value is **unknown**, **missing** or **not applicable**. It is NOT zero and NOT an empty string.

Use of NULL	Example
Value unknown	A student's phone number is not yet collected
Not applicable	Spouse name for an unmarried employee
Value exists but missing	Date of birth not recorded

- Any arithmetic with NULL gives NULL (**5 + NULL = NULL**).

- Comparison with NULL gives **UNKNOWN** (third truth value) - so use **IS NULL / IS NOT NULL**, never **= NULL**.
- Aggregates (SUM, AVG, COUNT(col)) ignore NULLs; **COUNT(*)** counts all rows.
- Primary key attributes can **never** be NULL.
- Importance: lets the DB store incomplete information without inventing fake values.

A	B	A AND B	A OR B	NOT A
TRUE	UNKNOWN	UNKNOWN	TRUE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN	TRUE
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Part 2 : Keys and Constraints

2.1 Keys

Key	Meaning	Example (STUDENT)
Super key	Any set of attributes that uniquely identifies a tuple	{Roll_no}, {Roll_no, Name}, {Email}
Candidate key	Minimal super key (no extra attribute)	Roll_no, Email
Primary key	Candidate key chosen by designer; not NULL; unique	Roll_no
Alternate key	Candidate keys not chosen as primary	Email
Composite key	Key made of two or more attributes	(Roll_no, Course_id) in ENROLLS
Foreign key	Attribute that refers to the primary key of another (or same) table	Dept_id in STUDENT

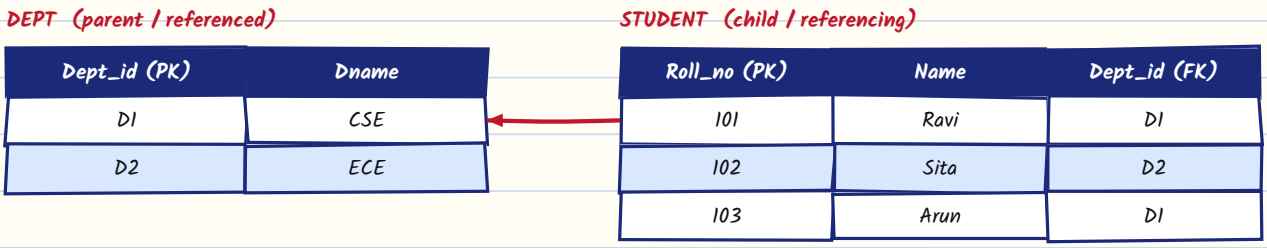


Fig 2.2 : Foreign key in STUDENT refers to primary key of DEPT

2.2 Integrity Constraints and their Importance

Constraints are rules that every instance of the database must obey. The DBMS checks them automatically on INSERT, UPDATE and DELETE, so wrong data never enters the database.

Constraint	Rule	Why important
Domain constraint	Value must come from the attribute's domain (data type, range, NOT NULL, CHECK)	Age cannot be -5 or 'abc'
Key constraint	Key attributes must be unique (UNIQUE / PRIMARY KEY)	No two students share a Roll_no
Entity integrity	Primary key cannot be NULL	Every row can be identified
Referential integrity	A foreign key value must either be NULL or match an existing primary key value in the parent table	No student belongs to a non-existing department

Actions to keep referential integrity (ON DELETE / ON UPDATE)

Option	Meaning
RESTRICT / NO ACTION	Reject the delete/update if child rows exist
CASCADE	Delete / update the child rows also
SET NULL	Set the foreign key in child rows to NULL
SET DEFAULT	Set the foreign key to its default value

* **Remember:** Entity integrity -> **primary key not null**. Referential integrity -> **foreign key matches primary key**. Domain -> **valid value**.

Part 3 : Relational Algebra

Relational algebra is a **procedural query language**: you tell the system step by step **how** to get the result. Each operator takes one or two relations and returns a new relation. It is the theoretical base of SQL.

Sample relations used below: STUDENT(Roll_no, Name, Age, Dept_id) with rows (101 Ravi 20 D1), (102 Sita 19 D2), (103 Arun 21 D1); DEPT(Dept_id, Dname) with rows (D1 CSE), (D2 ECE).

3.1 Fundamental (Basic) Operators

Operator	Symbol / syntax	Meaning and example
Select	$\sigma_{condition}(R)$	Chooses ROWS that satisfy the condition. $\sigma_{Age>19}(STUDENT)$ -> Ravi, Arun.
Project	$\pi_{A1,A2}(R)$	Chooses COLUMNS, removes duplicates. $\pi_{Name,Age}(STUDENT)$.
Union	$R \cup S$	All tuples in R or S or both (no duplicates). Needs union-compatible relations.
Set difference	$R - S$	Tuples in R but not in S.
Cartesian product	$R \times S$	Every tuple of R paired with every tuple of S; degree = sum, rows = m x n.
Rename	$\rho_X(R)$	Gives a new name to relation or attributes.

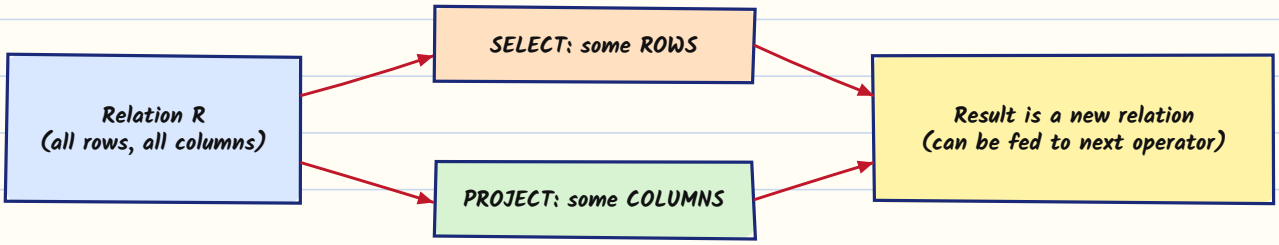


Fig 2.3 : Select picks rows, Project picks columns

* **Union-compatible:** Two relations must have the **same number of attributes** with **matching domains** for union, intersection and difference.

3.2 Additional Operators

Operator	Symbol	Meaning
Intersection	$R \cap S$	Tuples common to R and S. $R \cap S = R - (R - S)$.
Theta join	$R \bowtie_{\theta} S$	Cartesian product followed by select with condition θ (any comparison).
Equi-join	$R \bowtie_{R.A=S.B} S$	Theta join where condition uses only equality.
Natural join	$R \bowtie S$	Equi-join on all common attributes; the common column appears once.
Left / Right / Full outer join	$R \bowtie_{L} S$, $R \bowtie_{R} S$, $R \bowtie_{F} S$ (L = left, R = right, F = full)	Join that also keeps unmatched tuples of left / right / both, filling NULLs.
Division	$R \div S$	Tuples of R related to ALL tuples of S ("for all" queries).
Aggregate	$\mathcal{F}$ / G	SUM, AVG, MIN, MAX, COUNT, e.g. $_{Dept_id}\mathcal{G}_{COUNT(Roll_no)}(STUDENT)$.

Example: Natural join STUDENT ⋈ DEPT

Roll_no	Name	Age	Dept_id	Dname
101	Ravi	20	D1	CSE
102	Sita	19	D2	ECE
103	Arun	21	D1	CSE

Example queries

Query in English	Relational algebra
Names of students older than 19	$\pi_{Name}(\sigma_{Age>19}(STUDENT))$
Names of students of CSE	$\pi_{Name}(\sigma_{Dname='CSE'}(STUDENT \bowtie DEPT))$
Students in D1 and Age > 20	$\sigma_{Dept_id='D1' \wedge Age>20}(STUDENT)$
Departments with no students	$\pi_{Dept_id}(DEPT) - \pi_{Dept_id}(STUDENT)$

Exam tip: Select σ = **rows** (WHERE), Project π = **columns** (SELECT list). Only the six basic operators (σ, π, ∪, −, ×, ρ) are truly needed; others can be derived from them.

Part 4 : Relational Calculus

Relational calculus is a **non-procedural (declarative)** language based on predicate logic: you describe **what** result you want, not how to get it. Its expressive power equals relational algebra for safe expressions (Codd's theorem).

Point	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variable ranges over	Whole tuples (rows)	Individual attribute values (domains)
General form	$\{ t \mid P(t) \}$	$\{ \langle x1, x2, \ldots, xn \rangle \mid P(x1, \ldots, xn) \}$
Example: students with Age > 20	$\{ t \mid t \in STUDENT \wedge t.Age > 20 \}$	$\{ \langle r, n, a, d \rangle \mid \langle r, n, a, d \rangle \in STUDENT \wedge a > 20 \}$
Example: names only	$\{ t.Name \mid t \in STUDENT \wedge t.Age > 20 \}$	$\{ \langle n \rangle \mid \exists r,a,d\ (\langle r,n,a,d \rangle \in STUDENT \wedge a > 20) \}$
Basis of	SQL	QBE (Query-By-Example)

Symbols used

Symbol	Meaning	Symbol	Meaning
∧	AND	∃	there exists (at least one)
∨	OR	∀	for all
¬	NOT	⇒	implies

- **Free variable** - appears in the result; **bound variable** - quantified by ∃ or ∀.
- **Safe expression** - guarantees a finite result (only values from the database). Unsafe e.g. { t | ¬(t ∈ STUDENT) }.

Relational algebra	Relational calculus
Procedural - how to get result	Declarative - what result is needed
Uses operators on relations	Uses logic predicates and quantifiers
Order of operations matters	No order of operations
Base for query optimisation	Base for SQL (TRC) and QBE (DRC)

Part 5 : Basic SQL

5.1 What is SQL?

SQL (Structured Query Language) is the standard language of relational databases (ANSI / ISO). It is declarative and not case-sensitive for keywords. Statements end with a semicolon.

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define / change structure (schema)
DML	INSERT, UPDATE, DELETE (SELECT)	Change / read the data
DCL	GRANT, REVOKE	Access permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Control transactions

5.2 Simple Database Schema (used in examples)

```
DEPT    (Dept_id PK, Dname, Location)
STUDENT (Roll_no PK, Name, Age, Email, Dept_id FK -> DEPT, Fees)
COURSE  (Course_id PK, Title, Credits)
ENROLLS (Roll_no FK, Course_id FK, Grade)   PK = (Roll_no, Course_id)
```

5.3 Data Types

Type	Description	Example
INT / INTEGER	Whole numbers	101
SMALLINT / BIGINT	Small / large whole numbers	32000
DECIMAL(p,s) / NUMERIC	Exact number, p digits, s after point	DECIMAL(8,2) = 12345.67
FLOAT / REAL / DOUBLE	Approximate decimal number	3.14
CHAR(n)	Fixed length string (padded)	CHAR(2) = 'D1'
VARCHAR(n)	Variable length string up to n	'Ravi Kumar'

Type	Description	Example
TEXT	Very long text	address, notes
DATE / TIME / TIMESTAMP	Date, time, date+time	'2026-10-15'
BOOLEAN	TRUE / FALSE	TRUE
BLOB	Binary large object (image, file)	photo

5.4 Table Definition : CREATE

```
CREATE TABLE DEPT (
    Dept_id   CHAR(2)      PRIMARY KEY,
    Dname     VARCHAR(30)  NOT NULL UNIQUE,
    Location  VARCHAR(30)  DEFAULT 'Main Campus'
);

CREATE TABLE STUDENT (
    Roll_no  INT          PRIMARY KEY,
    Name     VARCHAR(40)   NOT NULL,
    Age      INT          CHECK (Age BETWEEN IS AND 60),
    Email    VARCHAR(50)   UNIQUE,
    Dept_id  CHAR(2),
    Fees     DECIMAL(8,2)  DEFAULT 0,
    FOREIGN KEY (Dept_id) REFERENCES DEPT(Dept_id)
        ON DELETE SET NULL ON UPDATE CASCADE
);
```

Column constraint	Effect
PRIMARY KEY	Unique + NOT NULL; identifies the row
NOT NULL	Value must be given
UNIQUE	No duplicate values (NULL allowed)
CHECK (cond)	Value must satisfy the condition (domain constraint)
DEFAULT v	Value used when none is supplied
FOREIGN KEY ... REFERENCES	Referential integrity link to another table

5.5 Table Definition : ALTER, DROP, TRUNCATE

```
ALTER TABLE STUDENT ADD Phone VARCHAR(15);          -- add column
ALTER TABLE STUDENT MODIFY Name VARCHAR(60);         -- change type (MySQL/Oracle)
ALTER TABLE STUDENT ALTER COLUMN Name TYPE VARCHAR(60);  -- PostgreSQL style
ALTER TABLE STUDENT DROP COLUMN Phone;               -- remove column
ALTER TABLE STUDENT ADD CONSTRAINT chk_age CHECK (Age >= 15);
ALTER TABLE STUDENT DROP CONSTRAINT chk_age;
ALTER TABLE STUDENT RENAME TO STUDENTS;              -- rename table

TRUNCATE TABLE STUDENT;   -- delete ALL rows, keep structure (fast, no rollback in most DBs)
DROP TABLE STUDENT;       -- delete structure + data permanently
```

Command	Removes	Structure kept?	WHERE allowed?
DELETE	Selected / all rows	Yes	Yes
TRUNCATE	All rows	Yes	No

Command	Removes	Structure kept?	WHERE allowed?
DROP	Whole table	No	No

5.6 DML Operations : INSERT, UPDATE, DELETE

INSERT – add new rows

```
INSERT INTO DEPT VALUES ('D1', 'CSE', 'Block A');
INSERT INTO STUDENT (Roll_no, Name, Age, Dept_id)      -- only some columns
      VALUES (101, 'Ravi', 20, 'D1');
INSERT INTO STUDENT VALUES (102,'Sita',19,'s@x.com','D2',5000);
INSERT INTO STUDENT (Roll_no, Name, Age)               -- many rows at once
      VALUES (103,'Arun',21), (104,'Meena',22);
INSERT INTO ALUMNI SELECT * FROM STUDENT WHERE Age > 22;   -- from a query
```

UPDATE – change existing rows

```
UPDATE STUDENT SET Age = 21 WHERE Roll_no = 101;
UPDATE STUDENT SET Fees = Fees * 1.10 WHERE Dept_id = 'D1';
UPDATE STUDENT SET Dept_id = 'D2', Age = Age + 1 WHERE Roll_no = 103;
```

DELETE – remove rows

```
DELETE FROM STUDENT WHERE Roll_no = 104;
DELETE FROM STUDENT WHERE Age < 18;
DELETE FROM STUDENT;          -- removes ALL rows (table remains)
```

* **Careful:** UPDATE or DELETE without a **WHERE** clause affects **every row**. Inserting a duplicate primary key, or a foreign key that does not exist, is rejected by the DBMS.

Quick Revision - Unit II

- Relational model (Codd, 1970): data in tables. Relation = table, tuple = row, attribute = column, domain = allowed values.
- Degree = number of columns; cardinality = number of rows.
- NULL = unknown / not applicable; comparisons give UNKNOWN; use IS NULL.
- Keys: super, candidate, primary, alternate, composite, foreign.
- Constraints: domain, key, entity integrity (PK not null), referential integrity (FK matches PK).
- Algebra (procedural): σ select rows, π project columns, ∪, −, ×, ρ basic; ∩, ⋈, ÷ derived.
- Calculus (declarative): TRC {t | P(t)} and DRC { | P}; ∃ and ∀.
- SQL: DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE), DCL, TCL.
- CREATE TABLE with PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY.
- DELETE removes rows, TRUNCATE removes all rows, DROP removes the table.

Possible exam questions

1. Explain the relational model with terms: domain, attribute, tuple, relation, degree, cardinality.
2. What are NULL values? Why are they important? How do they behave in comparisons?
3. Explain the different keys with examples.
4. Explain domain, key, entity integrity and referential integrity constraints and their importance.
5. Explain the operators of relational algebra with examples.
6. Differentiate tuple and domain relational calculus; algebra vs calculus.
7. Write the SQL statements to create tables with constraints, and alter them.
8. Explain INSERT, UPDATE and DELETE with examples. Differentiate DELETE, TRUNCATE, DROP.

DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT - III : SQL (Structured Query Language)

Easy handwritten notes with diagrams for students

Topics in this unit

1. Basic SQL querying: SELECT, project, WHERE, operators
2. SQL functions: date and time, numeric, string, conversion
3. Creating tables with relationships; key and integrity constraints
4. Grouping, aggregation, HAVING and ordering
5. Joins (inner, outer, cross, self)
6. Nested queries and sub queries
7. Views (updatable and non-updatable)
8. Relational set operations

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions (Date and Time, Numeric, String conversion). Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view (updatable and non-updatable), relational set operations.

Part I : Basic SQL Querying

All examples use these tables (from Unit II):

```
DEPT    (Dept_id, Dname, Location)
STUDENT (Roll_no, Name, Age, Email, Dept_id, Fees)
COURSE  (Course_id, Title, Credits)
ENROLLS (Roll_no, Course_id, Grade)
```

1.1 SELECT statement (Select = Project)

SELECT retrieves data. The column list is the relational-algebra **projection** (π) and the **WHERE** clause is the **selection** (σ).

```
SELECT column_list | *          -- what columns (projection)
FROM   table_list               -- from which tables
WHERE  condition                -- which rows (selection)
GROUP BY columns                -- make groups
HAVING group_condition          -- which groups
ORDER BY columns [ASC|DESC]     -- sort result
LIMIT n;                        -- first n rows (MySQL/PostgreSQL)
```

```
SELECT * FROM STUDENT;                          -- all columns, all rows
SELECT Name, Age FROM STUDENT;                  -- project two columns
SELECT DISTINCT Dept_id FROM STUDENT;           -- remove duplicates
SELECT Name AS Student_Name, Fees/12 AS Monthly FROM STUDENT;   -- alias + arithmetic
SELECT Name FROM STUDENT WHERE Age > 20;        -- select rows
```

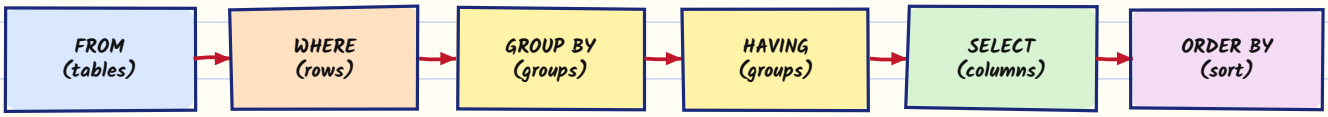


Fig 3.1 : Logical order in which SQL executes a query (not the order it is written)

1.2 WHERE clause - Arithmetic, Comparison and Logical operators

Type	Operators	Example
Arithmetic	+ - * / % (MOD)	SELECT Name, Fees*1.1 FROM STUDENT;
Comparison	= <> (or !=) < > <= >=	WHERE Age >= 20
Logical	AND OR NOT	WHERE Age > 18 AND Dept_id = 'D1'
Range	BETWEEN a AND b (inclusive)	WHERE Age BETWEEN 18 AND 22
List	IN (v1, v2...) / NOT IN	WHERE Dept_id IN ('D1','D2')
Pattern	LIKE with % (any string), _ (one char)	WHERE Name LIKE 'R%' / '_a%'
Null test	IS NULL / IS NOT NULL	WHERE Email IS NULL

* **Precedence:** ( ) first, then * / %, then + -, then comparison, then **NOT**, then **AND**, then **OR**. Use brackets to be safe.

```
SELECT Name FROM STUDENT WHERE Name LIKE 'A%';      -- starts with A
SELECT Name FROM STUDENT WHERE Name LIKE '%n';      -- ends with n
SELECT Name FROM STUDENT WHERE Name LIKE '_a%';     -- second letter a
```

```
SELECT * FROM STUDENT WHERE (Age < 19 OR Age > 21) AND Dept_id = 'D1';
```

1.3 ORDER BY and LIMIT

```
SELECT Name, Fees FROM STUDENT ORDER BY Fees DESC;          -- highest first
SELECT * FROM STUDENT ORDER BY Dept_id ASC, Age DESC;       -- multi-column
SELECT * FROM STUDENT ORDER BY Fees DESC LIMIT 3;           -- top 3 (Oracle: FETCH FIRST 3 ROWS ONLY)
```

ASC (ascending) is the default. NULLs sort first or last depending on the DBMS.

Part 2 : SQL Functions

Functions take values and return a value. They are of two kinds: **single-row functions** (one result per row - below) and **aggregate functions** (one result per group - Part 4). Names differ slightly between MySQL, Oracle and PostgreSQL.

2.1 Numeric functions

Function	Purpose	Example -> Result
ABS(x)	Absolute value	ABS(-7) -> 7
ROUND(x, d)	Round to d decimals	ROUND(15.678, 1) -> 15.7
CEIL(x) / FLOOR(x)	Round up / down	CEIL(4.2) -> 5 ; FLOOR(4.8) -> 4
MOD(x, y)	Remainder	MOD(10, 3) -> 1
POWER(x, y)	x raised to y	POWER(2, 5) -> 32
SQRT(x)	Square root	SQRT(49) -> 7
TRUNC(x, d) / TRUNCATE	Cut decimals (no rounding)	TRUNC(15.79, 1) -> 15.7
SIGN(x)	-1, 0 or 1	SIGN(-5) -> -1

2.2 String functions

Function	Purpose	Example -> Result
UPPER(s) / LOWER(s)	Change case	UPPER('ravi') -> RAVI
LENGTH(s)	Number of characters	LENGTH('Sita') -> 4
CONCAT(a, b) or a || b	Join strings	CONCAT('Data','base') -> Database
SUBSTR(s, start, len)	Part of a string	SUBSTR('DATABASE', 1, 4) -> DATA
INSTR(s, sub)	Position of substring	INSTR('DATABASE','BASE') -> 5
TRIM(s) / LTRIM / RTRIM	Remove spaces	TRIM(' hi ') -> hi
REPLACE(s, old, new)	Replace text	REPLACE('cat','c','b') -> bat
LPAD(s, n, ch) / RPAD	Pad to length n	LPAD('7', 3, '0') -> 007
REVERSE(s)	Reverse string	REVERSE('abc') -> cba

2.3 Date and Time functions

Function	Purpose	Example
CURRENT_DATE / CURDATE()	Today's date	SELECT CURRENT_DATE;

Function	Purpose	Example
NOW() / SYSDATE / CURRENT_TIMESTAMP	Current date + time	SELECT NOW();
EXTRACT(part FROM d)	Year / month / day of a date	EXTRACT(YEAR FROM DOB)
YEAR(d), MONTH(d), DAY(d)	Parts (MySQL)	YEAR('2026-10-15') -> 2026
DATE_ADD(d, INTERVAL n unit)	Add time to date	DATE_ADD(d, INTERVAL 7 DAY)
ADD_MONTHS(d, n)	Add months (Oracle)	ADD_MONTHS(SYSDATE, 3)
DATEDIFF(d1, d2)	Days between dates	DATEDIFF('2026-10-15','2026-10-01') -> 14
LAST_DAY(d)	Last date of the month	LAST_DAY('2026-02-10') -> 2026-02-28
DATE_FORMAT(d, fmt) / TO_CHAR(d, fmt)	Display in a format	DATE_FORMAT(d,'%d-%m-%Y')

2.4 Conversion functions

Function	Purpose	Example
CAST(x AS type)	Standard type conversion	CAST('123' AS INT) -> 123
CONVERT(x, type)	MySQL / SQL Server conversion	CONVERT('2026-10-15', DATE)
TO_CHAR(x, fmt)	Number/date -> text (Oracle, PostgreSQL)	TO_CHAR(SYSDATE,'DD-MON-YYYY')
TO_NUMBER(s)	Text -> number	TO_NUMBER('45.5') -> 45.5
TO_DATE(s, fmt)	Text -> date	TO_DATE('15-10-2026','DD-MM-YYYY')
COALESCE(a, b, ...) / NVL(a, b)	First non-NULL value	COALESCE(Email, 'none')

```
SELECT UPPER(Name), ROUND(Fees/3, 2), LENGTH(Name) FROM STUDENT;
SELECT Name, COALESCE(Email, 'not given') FROM STUDENT;      -- replace NULL
SELECT Name, CAST(Fees AS INT) FROM STUDENT;
```

Part 3 : Creating Tables with Relationships; Constraints

Relationships from the ER diagram are implemented using **primary keys** and **foreign keys**.

Relationship	How to implement	Example
One-to-One (1:1)	FK in either table + UNIQUE (or share the same PK)	PERSON - PASSPORT: Passport.Person_id UNIQUE FK
One-to-Many (1:N)	Put FK on the 'many' side referring to the 'one' side	DEPT (1) - STUDENT (N): STUDENT.Dept_id FK
Many-to-Many (M:N)	New **junction (bridge) table** with FKs of both; PK = both FKs	STUDENT - COURSE -> ENROLLS(Roll_no, Course_id)

```
-- 1:N  (one department, many students)
CREATE TABLE STUDENT (
    Roll_no   INT PRIMARY KEY,
    Name      VARCHAR(40) NOT NULL,
    Dept_id   CHAR(2),
    CONSTRAINT fk_dept FOREIGN KEY (Dept_id) REFERENCES DEPT(Dept_id)
);
```

```
-- M:N  (junction table)
CREATE TABLE ENROLLS (
    Roll_no    INT,
    Course_id  VARCHAR(6),
    Grade      CHAR(1) CHECK (Grade IN ('A','B','C','D','F')),
    PRIMARY KEY (Roll_no, Course_id),                    -- composite key
    FOREIGN KEY (Roll_no)   REFERENCES STUDENT(Roll_no) ON DELETE CASCADE,
    FOREIGN KEY (Course_id) REFERENCES COURSE(Course_id) ON DELETE CASCADE
);
```

3.1 Implementing key and integrity constraints

Constraint	Implemented by	Enforces
Entity integrity	PRIMARY KEY	Unique, not NULL row identity
Key / uniqueness	UNIQUE	No duplicate values in a column
Domain	data type, NOT NULL, CHECK, DEFAULT	Valid values only
Referential integrity	FOREIGN KEY ... REFERENCES	Child value must exist in parent
Referential actions	ON DELETE / ON UPDATE CASCADE, SET NULL, RESTRICT	What happens to child rows
General rules	CHECK (condition), ASSERTION / TRIGGER	Business rules

```
ALTER TABLE STUDENT ADD CONSTRAINT uq_email UNIQUE (Email);
ALTER TABLE STUDENT ADD CONSTRAINT ck_age  CHECK (Age BETWEEN IS AND 60);
ALTER TABLE STUDENT DROP CONSTRAINT uq_email;
-- Violations are rejected, e.g.:
INSERT INTO STUDENT (Roll_no, Name, Dept_id) VALUES (101, 'X', 'D9');   -- ERROR: D9 not in DEPT
```

Part 4 : Aggregation, Grouping and HAVING

4.1 Aggregate functions

Function	Returns	Notes
COUNT(*)	Number of rows	Counts NULLs too
COUNT(col)	Number of non-NULL values	COUNT(DISTINCT col) for unique values
SUM(col)	Total	Ignores NULL
AVG(col)	Average	Ignores NULL
MIN(col) / MAX(col)	Smallest / largest	Works on numbers, text, dates

```
SELECT COUNT(*) FROM STUDENT;                    -- total students
SELECT AVG(Fees), MAX(Age), MIN(Age) FROM STUDENT;
```

4.2 GROUP BY and HAVING

GROUP BY divides rows into groups with the same value; the aggregate is computed for each group. **HAVING** filters groups (after grouping); **WHERE** filters rows (before grouping).

```
SELECT Dept_id, COUNT(*) AS Total, AVG(Fees) AS Avg_fee
FROM   STUDENT
```

```
WHERE  Age >= 18                -- filter ROWS first
GROUP  BY Dept_id               -- one row per department
HAVING COUNT(*) > 2              -- filter GROUPS
ORDER  BY Total DESC;
```

WHERE	HAVING
Filters individual rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions	Can use aggregate functions

* **Rule:** Every column in SELECT that is **not inside an aggregate** must appear in the **GROUP BY** clause.

Part 5 : Joins

A **join** combines rows of two or more tables using a related column (usually PK-FK).

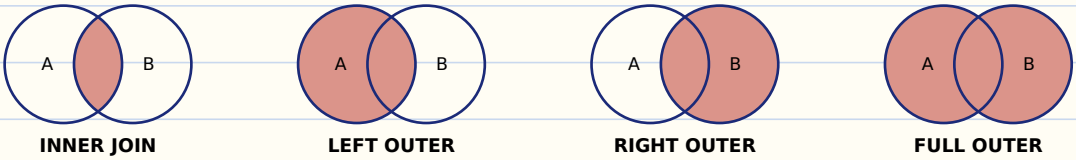


Fig 3.2 : Joins as Venn diagrams (shaded part = rows returned)

Join	Result	Syntax
CROSS JOIN	Cartesian product: every row with every row (m x n)	FROM A CROSS JOIN B
INNER JOIN	Only matching rows from both tables	FROM A INNER JOIN B ON A.x = B.x
NATURAL JOIN	Inner join on all columns with the same name	FROM A NATURAL JOIN B
LEFT OUTER JOIN	All rows of A + matching rows of B (NULL if none)	FROM A LEFT JOIN B ON ...
RIGHT OUTER JOIN	All rows of B + matching rows of A	FROM A RIGHT JOIN B ON ...
FULL OUTER JOIN	All rows of both; NULL where no match	FROM A FULL JOIN B ON ...
SELF JOIN	A table joined with itself using aliases	FROM EMP E1 JOIN EMP E2 ON E1.Mgr = E2.Id

```
-- INNER: student names with department names
SELECT S.Name, D.Dname
FROM   STUDENT S INNER JOIN DEPT D ON S.Dept_id = D.Dept_id;

-- LEFT OUTER: all departments, even those with no students
SELECT D.Dname, S.Name
FROM   DEPT D LEFT JOIN STUDENT S ON D.Dept_id = S.Dept_id;

-- Join three tables (M:N)
SELECT S.Name, C.Title, E.Grade
FROM   STUDENT S JOIN ENROLLS E ON S.Roll_no = E.Roll_no
               JOIN COURSE  C ON E.Course_id = C.Course_id;
```

```
-- SELF JOIN: employee with manager name
SELECT E.Name AS Employee, M.Name AS Manager
FROM   EMP E LEFT JOIN EMP M ON E.Mgr_id = M.Emp_id;
```

* **Remember:** Use **ON** for join conditions and **WHERE** for row filters. Forgetting the join condition gives a **cross product** with wrong, huge results.

Part 6 : Nested Queries and Sub Queries

A **subquery** is a SELECT inside another query (in parentheses). The inner query runs first and passes its result to the outer query.

Type	Description	Example
Scalar subquery	Returns ONE value; used with = < >	WHERE Fees > (SELECT AVG(Fees) FROM STUDENT)
Multi-row (IN)	Returns a list; used with IN / NOT IN / ANY / ALL	WHERE Dept_id IN (SELECT Dept_id FROM DEPT WHERE Location='Block A')
Correlated	Inner query uses a column of the outer query; runs once per outer row	WHERE EXISTS (SELECT 1 FROM ENROLLS E WHERE E.Roll_no = S.Roll_no)
FROM subquery	Result used as a temporary table (needs alias)	FROM (SELECT Dept_id, AVG(Fees) a FROM STUDENT GROUP BY Dept_id) T

```
-- students paying more than the average fee
SELECT Name FROM STUDENT WHERE Fees > (SELECT AVG(Fees) FROM STUDENT);

-- students of departments located in Block A
SELECT Name FROM STUDENT
WHERE Dept_id IN (SELECT Dept_id FROM DEPT WHERE Location = 'Block A');

-- students who are enrolled in at least one course   (EXISTS)
SELECT Name FROM STUDENT S
WHERE EXISTS (SELECT * FROM ENROLLS E WHERE E.Roll_no = S.Roll_no);

-- students NOT enrolled in any course
SELECT Name FROM STUDENT
WHERE Roll_no NOT IN (SELECT Roll_no FROM ENROLLS);

-- student(s) with the highest fee      (ALL)
SELECT Name FROM STUDENT WHERE Fees >= ALL (SELECT Fees FROM STUDENT);

-- 2nd highest fee (nested)
SELECT MAX(Fees) FROM STUDENT WHERE Fees < (SELECT MAX(Fees) FROM STUDENT);
```

Operator	Meaning
IN / NOT IN	Value is (not) in the subquery result list
ANY / SOME	Condition true for at least one value of the list
ALL	Condition true for every value of the list
EXISTS / NOT EXISTS	True if the subquery returns at least one row (any)

* **Beware:** **NOT IN** returns no rows if the subquery list contains a **NULL**. Prefer **NOT EXISTS** in such cases.

Part 7 : Relational Set Operations

Set operations combine the results of two SELECT queries. Both queries must have the **same number of columns** with **compatible data types**. They remove duplicates by default.

Operation	Result	Algebra
UNION	All rows of both queries, duplicates removed (UNION ALL keeps duplicates)	$R \cup S$
INTERSECT	Rows present in both queries	$R \cap S$
EXCEPT (MINUS in Oracle)	Rows of first query not in second	$R - S$

```
SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C1'
UNION                                  -- enrolled in C1 OR C2
SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C2';

SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C1'
INTERSECT                               -- enrolled in C1 AND C2
SELECT Roll_no FROM ENROLLS WHERE Course_id = 'C2';

SELECT Roll_no FROM STUDENT
EXCEPT                                 -- students with NO enrolment
SELECT Roll_no FROM ENROLLS;
```

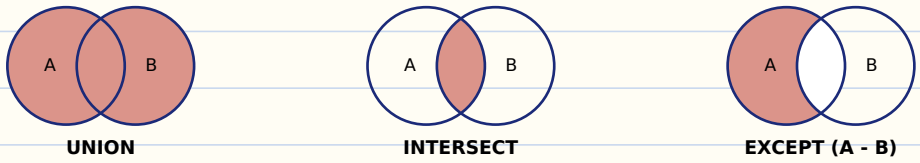


Fig 3.3 : Set operations on two query results A and B

Part 8 : Views

A **view** is a **virtual table** defined by a stored SELECT query. It holds no data itself; the query runs each time the view is used. Uses: **security** (hide columns / rows), **simplicity** (hide complex joins), **logical data independence**.

```
CREATE VIEW CSE_STUDENTS AS
    SELECT Roll_no, Name, Age FROM STUDENT WHERE Dept_id = 'D1';

SELECT * FROM CSE_STUDENTS WHERE Age > 19;          -- use like a table
CREATE OR REPLACE VIEW CSE_STUDENTS AS SELECT ... ;  -- change definition
DROP VIEW CSE_STUDENTS;                              -- remove view (data stays)
```

Updatable and non-updatable views

An **updatable view** allows INSERT / UPDATE / DELETE, which are passed to the base table. A **non-updatable (read-only) view** allows only SELECT.

Updatable view (simple)	Non-updatable view if it has ...
Based on ONE base table	Joins of several tables (usually)

Updatable view (simple)	Non-updatable view if it has ...
Includes the primary key	DISTINCT
No aggregate functions	Aggregate functions (SUM, AVG, COUNT ...)
No GROUP BY / HAVING	GROUP BY or HAVING
All NOT NULL base columns present	Set operations (UNION etc.) or subquery in FROM
Columns are real columns, not expressions	Computed / derived columns

```
CREATE VIEW ADULTS AS
    SELECT Roll_no, Name, Age FROM STUDENT WHERE Age >= 18
    WITH CHECK OPTION;       -- rejects updates that would make a row disappear from the view

UPDATE ADULTS SET Name = 'Ravi K' WHERE Roll_no = 101;    -- OK (updates STUDENT)
UPDATE ADULTS SET Age = 15 WHERE Roll_no = 101;           -- ERROR due to CHECK OPTION
```

* **Remember:** Views = **virtual tables**; **materialized views** store the result physically and must be refreshed.

Quick Revision - Unit III

- SELECT (projection) ... FROM ... WHERE (selection) ... GROUP BY ... HAVING ... ORDER BY.
- Operators: arithmetic, comparison, logical (AND/OR/NOT), BETWEEN, IN, LIKE (% and _), IS NULL.
- Functions: numeric (ROUND, CEIL, MOD), string (UPPER, SUBSTR, CONCAT), date (NOW, DATEDIFF, EXTRACT), conversion (CAST, TO_CHAR, TO_DATE).
- 1:N = FK on many side; M:N = junction table with composite PK; constraints PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT.
- Aggregates: COUNT, SUM, AVG, MIN, MAX. WHERE filters rows, HAVING filters groups.
- Joins: inner, left/right/full outer, cross, natural, self.
- Subqueries: scalar, IN, ANY/ALL, EXISTS, correlated, FROM-subquery.
- Set operations: UNION, INTERSECT, EXCEPT / MINUS (same columns needed).
- View = virtual table; updatable only if simple (one table, no aggregate / DISTINCT / GROUP BY).

Possible exam questions

1. Explain the SELECT statement with WHERE, GROUP BY, HAVING and ORDER BY with examples.
2. Explain the numeric, string, date/time and conversion functions in SQL.
3. How are 1:1, 1:N and M:N relationships implemented? Show SQL for tables with constraints.
4. Differentiate WHERE and HAVING. Explain aggregate functions.
5. Explain different types of joins with examples.
6. What are nested queries / correlated subqueries? Explain IN, EXISTS, ANY, ALL.
7. Explain UNION, INTERSECT and EXCEPT.
8. What is a view? Explain updatable and non-updatable views with examples.

DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT - IV : Schema Refinement (Normalization)

Easy handwritten notes with diagrams for students

Topics in this unit

1. Purpose of normalization and anomalies
2. Functional dependency, Armstrong's axioms, closure
3. Lossless-join and dependency-preserving decomposition
4. INF, 2NF, 3NF and surrogate key
5. Boyce-Codd Normal Form (BCNF)
6. Multivalued dependency and 4NF
7. Join dependency and 5NF

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Schema Refinement (Normalization): Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (INF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form (BCNF), MVD, Fourth normal form (4NF), Fifth Normal Form (5NF).

Part 1 : Purpose of Normalization

1.1 What is Normalization?

Definition: **Normalization** (schema refinement) is the process of organising the tables of a database to **reduce redundancy** and avoid **anomalies** by breaking a large table into smaller, well-structured tables linked by keys. It uses **functional dependencies** and a series of **normal forms**.

Bad design example - all facts in ONE table:

Emp_id	Emp_name	Dept_id	Dept_name	Dept_head
E1	Ravi	D1	CSE	Dr. Rao
E2	Sita	D1	CSE	Dr. Rao
E3	Arun	D2	ECE	Dr. Iyer
E4	Meena	D1	CSE	Dr. Rao

Department data (D1, CSE, Dr. Rao) is repeated for every employee. This **redundancy** causes three kinds of anomalies:

Anomaly	Problem	Example above
Insertion anomaly	Cannot insert some data without other data	Cannot add a new department D3 until an employee joins it
Deletion anomaly	Deleting one fact loses another fact	Deleting Arun (E3) also deletes the only record of ECE and Dr. Iyer
Update (modification) anomaly	Same fact stored many times; update must change all copies	Changing CSE's head needs 3 row updates; missing one gives inconsistency

Goals of normalization

- Remove **redundant data** and save space.
- Remove **insert / update / delete anomalies**.
- Keep data **consistent** and make the design easier to maintain.
- Decompose tables without losing information (lossless) and, if possible, without losing dependencies.

* **Trade-off:** Too much normalization gives many tables and more joins (slower reads). Sometimes designers do **denormalization** on purpose for speed.

Part 2 : Functional Dependency (FD)

2.1 Concept

Definition: A **functional dependency** X -> Y (read "X determines Y") means: if two tuples have the same value for X, they must have the same value for Y. X is the **determinant** (left side), Y is the **dependent** (right side).

Examples: **Roll_no -> Name**, **Dept_id -> Dept_name**, **(Roll_no, Course_id) -> Grade**. But **Name -> Roll_no** is not valid, because two students can have the same name.

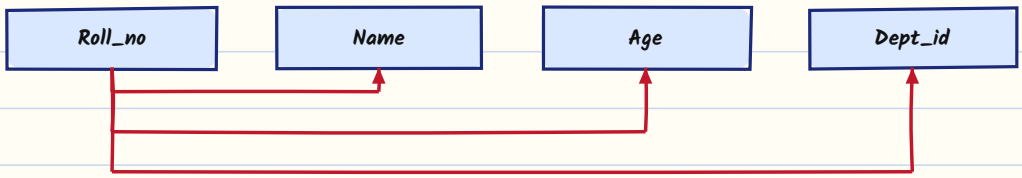


Fig 4.1 : FD diagram - Roll_no determines every other attribute (so Roll_no is a key)

Types of functional dependency

Type	Meaning	Example
Trivial	Y is a subset of X (always true)	{Roll_no, Name} -> Name
Non-trivial	Y is not a subset of X	Roll_no -> Name
Completely non-trivial	X and Y have no common attribute	Roll_no -> Name
Full dependency	Y depends on the whole of X (no proper subset of X is enough)	(Roll_no, Course_id) -> Grade
Partial dependency	Y depends on only PART of a composite key	(Roll_no, Course_id) -> Name, because Roll_no -> Name
Transitive dependency	X -> Y and Y -> Z (Y not a key) so X -> Z indirectly	Emp_id -> Dept_id -> Dept_name

2.2 Armstrong's Axioms (inference rules)

Rule	Statement
Reflexivity	If Y is a subset of X, then X -> Y
Augmentation	If X -> Y, then XZ -> YZ for any Z
Transitivity	If X -> Y and Y -> Z, then X -> Z
Union (derived)	If X -> Y and X -> Z, then X -> YZ
Decomposition (derived)	If X -> YZ, then X -> Y and X -> Z
Pseudo-transitivity (derived)	If X -> Y and WY -> Z, then WX -> Z

Armstrong's axioms are **sound** (they derive only valid FDs) and **complete** (they derive all valid FDs).

2.3 Closure of a Set of Attributes (X^+)

The closure X^+ is the set of all attributes that X determines. Algorithm: start with result = X; repeatedly add Y for every FD A -> Y where A is already inside result; stop when nothing new is added.

Example

R(A, B, C, D, E) with F = { A -> B, B -> C, CD -> E }

Step	Result	Reason
Start	{A, D}	X = AD
A -> B	{A, D, B}	A is in result, add B
B -> C	{A, D, B, C}	B is in result, add C
CD -> E	{A, B, C, D, E}	C and D in result, add E

So $(AD)^+$ = all attributes of R, hence **AD is a candidate key**. $A^+ = \{A,B,C\}$ and $D^+ = \{D\}$, so no smaller subset works.

Uses of closure

- Find **candidate keys**: X is a super key if X^+ contains all attributes of R.
- Test whether an FD X -> Y holds: check if Y is inside X^+.
- Check equivalence / cover of FD sets (canonical or minimal cover: single attribute on right, no extra left attribute, no redundant FD).

> * **Tip to find keys fast:** Attributes that never appear on the **right side** of any FD must be part of every key. Start with them and compute the closure.

Part 3 : Decomposition

Decomposition replaces a relation R by two or more smaller relations R1, R2, ... such that together they hold the same information. A good decomposition must be **lossless** and preferably **dependency preserving**.

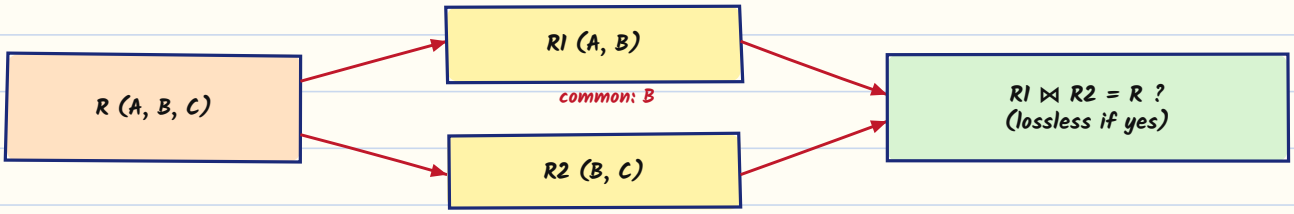


Fig 4.2 : Decomposition and re-joining

3.1 Lossless-Join Decomposition

After natural join of the parts we must get back **exactly the original relation** - no missing and no extra (spurious) tuples.

> **Definition:** Decomposition of R into R1 and R2 is **lossless** if and only if at least one of these holds in F^+: (R1 ∩ R2) -> R1 or (R1 ∩ R2) -> R2. That is, the **common attribute(s) must be a key** of at least one of the two parts.

Example: R(Emp_id, Dept_id, Dept_name) with Dept_id -> Dept_name. Split into R1(Emp_id, Dept_id) and R2(Dept_id, Dept_name). Common = Dept_id, and Dept_id -> Dept_name (key of R2) so it is **lossless**.

Bad split: R1(Emp_id, Dept_name) and R2(Dept_id, Dept_name) - common attribute Dept_name is a key of neither, so joining gives **spurious tuples** (lossy).

3.2 Dependency-Preserving Decomposition

Every FD of the original relation should be checkable inside a single decomposed table, without joining. Formally $(F1 \cup F2 \cup ...)^+ = F^+$.

Property	Why needed	Must always hold?
Lossless join	No information is lost or invented	**Yes - mandatory**
Dependency preservation	Constraints can be checked cheaply, without joins	Desirable (BCNF may not give it)

Part 4 : Normal Forms based on Functional Dependencies



Fig 4.3 : Each higher normal form includes (is stricter than) the previous one

Normal form	Removes	Condition (short)
1NF	Repeating groups / multi-valued attributes	All values atomic
2NF	Partial dependency	1NF + no non-key attribute depends on part of a key
3NF	Transitive dependency	2NF + no non-key attribute depends on another non-key attribute
BCNF	Anomalies from FDs whose determinant is not a key	For every FD X -> Y, X is a super key
4NF	Non-trivial multivalued dependencies	BCNF + for every MVD X ->> Y, X is a super key
5NF	Join dependencies not implied by keys	4NF + every join dependency is implied by candidate keys

4.1 First Normal Form (1NF)

A relation is in **1NF** if every attribute holds only **atomic (single, indivisible) values** - no multi-valued attributes, no repeating groups, no nested tables.

Not in 1NF:

Roll_no	Name	Phone
101	Ravi	98480, 98490
102	Sita	99000

Converted to 1NF (one value per cell; key becomes (Roll_no, Phone)):

Roll_no	Name	Phone
101	Ravi	98480
101	Ravi	98490
102	Sita	99000

Better: keep STUDENT(Roll_no, Name) and a separate PHONE(Roll_no, Phone) table.

4.2 Second Normal Form (2NF)

A relation is in **2NF** if it is in 1NF and has **no partial dependency**: every non-key attribute depends on the **whole** primary key. (A table with a single-attribute key is automatically in 2NF.)

ENROLL(Roll_no, Course_id, Student_name, Course_title, Grade) with key (Roll_no, Course_id)

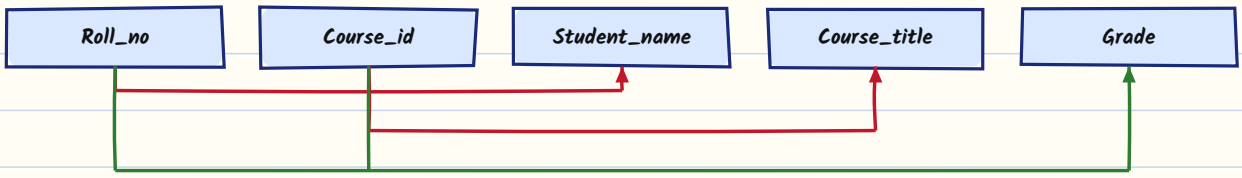


Fig 4.4 : Red = partial dependencies (bad); green = full dependency (good)

Decompose to 2NF:

- STUDENT(Roll_no, Student_name)
- COURSE(Course_id, Course_title)
- ENROLL(Roll_no, Course_id, Grade)

4.3 Third Normal Form (3NF)

A relation is in **3NF** if it is in 2NF and has **no transitive dependency**. Formally, for every non-trivial FD X -> A, either X is a **super key** or A is a **prime attribute** (part of some candidate key).

EMP(Emp_id, Emp_name, Dept_id, Dept_name) with Emp_id -> Dept_id and Dept_id -> Dept_name

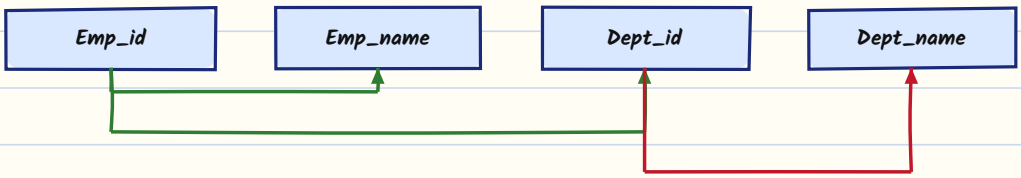


Fig 4.5 : Dept_id -> Dept_name (red) is a transitive dependency: Emp_id -> Dept_id -> Dept_name

Decompose to 3NF: EMP(Emp_id, Emp_name, Dept_id) and DEPT(Dept_id, Dept_name).

> * **3NF is always possible:** Every relation can be decomposed into 3NF with a **lossless join and dependency preservation** (3NF synthesis algorithm).

4.4 Surrogate Key

> **Definition:** A **surrogate key** is an artificial, system-generated identifier (usually an integer such as AUTO_INCREMENT / SERIAL / IDENTITY / sequence) with **no business meaning**, used as the primary key instead of a natural key.

	Natural key	Surrogate key
Source	Real-world data (Email, Aadhaar, ISBN)	Generated by DBMS (1, 2, 3 ...)
Meaning	Has meaning for the business	No meaning
Changes	May change (email, name change)	Never changes
Size	Often long or composite	Short, single integer; fast joins
Example	Roll_no = '23A05402'	Student_sk = 1007

- Use when no good natural key exists, the natural key is composite / long / changeable.
- Disadvantages: extra column, user cannot recognise rows; still keep a UNIQUE constraint on the natural key.

- Example: **CREATE TABLE STUDENT (Student_sk INT AUTO_INCREMENT PRIMARY KEY, Roll_no VARCHAR(10) UNIQUE, Name VARCHAR(40));**

4.5 Boyce-Codd Normal Form (BCNF)

> **Definition:** A relation is in **BCNF** if for **every non-trivial FD X -> Y, X is a super key**. BCNF is slightly stronger than 3NF (3NF allows Y prime; BCNF does not).

Example: TEACH(Student, Subject, Teacher) with FDs: {Student, Subject} -> Teacher; Teacher -> Subject (each teacher teaches only one subject).

- Candidate keys: (Student, Subject) and (Student, Teacher).
- Teacher -> Subject: Teacher is **not a super key** -> violates BCNF. But Subject is a prime attribute, so the table is in 3NF.

Decompose to BCNF: R1(Teacher, Subject) and R2(Student, Teacher). The join is lossless (common attribute Teacher is a key of R1), but the FD {Student, Subject} -> Teacher is **no longer preserved**.

Point	3NF	BCNF
Condition	X is super key OR Y is prime	X must be a super key
Redundancy	Small redundancy may remain	Removes all FD-based redundancy
Lossless join	Always possible	Always possible
Dependency preservation	Always possible	Not always possible

> **Exam tip:** Every BCNF relation is in 3NF, but not every 3NF relation is in BCNF. Order: 1NF ⊃ 2NF ⊃ 3NF ⊃ BCNF ⊃ 4NF ⊃ 5NF (each is a subset of the previous).

Part 5 : Multivalued Dependency and 4NF

5.1 Multivalued Dependency (MVD)

> **Definition:** An MVD X ->> Y (X multi-determines Y) holds when each value of X is associated with a **set of values of Y**, and this set is **independent** of the other attributes of the relation. It arises when a table holds two or more **independent multi-valued facts** about the same entity.

COURSE_INFO(Course, Teacher, Book): a course can have several teachers and several books, independently.

Course	Teacher	Book
DBMS	Rao	Korth
DBMS	Rao	Navathe
DBMS	Iyer	Korth
DBMS	Iyer	Navathe

Here Course ->> Teacher and Course ->> Book. Every teacher is combined with every book, causing redundancy even though the table is in BCNF (only trivial FDs, key = all three attributes).

- Every FD is also an MVD (X -> Y implies X ->> Y).
- MVD rules: complementation (X ->> Y implies X ->> R-X-Y), augmentation, transitivity.
- Trivial MVD: Y is a subset of X, or X ∪ Y = all attributes of R.

5.2 Fourth Normal Form (4NF)

A relation is in **4NF** if it is in BCNF and has **no non-trivial multivalued dependency** unless the determinant is a super key. Decompose by separating the independent multi-valued facts:

COURSE_TEACHER	
Course	Teacher
DBMS	Rao
DBMS	Iyer

COURSE_BOOK	
Course	Book
DBMS	Korth
DBMS	Navathe

4 rows become 2 + 2 rows; adding a new teacher now needs only **one** row instead of one per book.

Part 6 : Join Dependency and SNF

6.1 Join Dependency (JD)

A relation R satisfies a **join dependency** *(R1, R2, ..., Rn) if R can be split into R1 ... Rn and rebuilt exactly by their natural join. An MVD is a special JD with two parts (n = 2).

6.2 Fifth Normal Form (SNF / PJNF)

> **Definition:** A relation is in **SNF** (Project-Join Normal Form) if it is in 4NF and every join dependency in it is implied by its **candidate keys**. In simple words: the table cannot be split into smaller tables **without information loss** unless the split is based on keys.

Example: SPJ(Supplier, Part, Project) - a supplier supplies a part to a project only if (supplier supplies part) AND (supplier works for project) AND (project uses part). The rule is a 3-way constraint, so the table can be split into three binary tables:

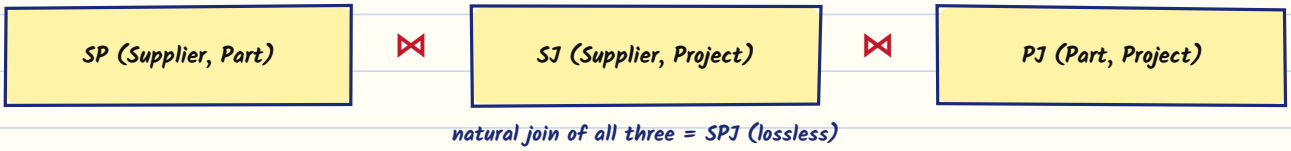


Fig 4.6 : SNF decomposition of SPJ into three tables

Normal form	Dependency it deals with	Typical problem solved
1NF	Atomic values	Multi-valued cells
2NF	Partial FD	Dependence on part of key
3NF	Transitive FD	Non-key determines non-key
BCNF	FD with non-key determinant	Remaining FD anomalies
4NF	Multivalued dependency	Independent multi-valued facts
5NF	Join dependency	Rare 3-way constraints

Quick Revision - Unit IV

- Normalization removes redundancy and insert / update / delete anomalies.
- FD X -> Y: same X means same Y. Types: trivial, non-trivial, full, partial, transitive.
- Armstrong's axioms: reflexivity, augmentation, transitivity. Closure X+ finds keys and checks FDs.
- Lossless join: common attribute must be a key of R1 or R2. Dependency preserving: (F1 ∪ F2)+ = F+.
- 1NF: atomic values. 2NF: no partial dependency. 3NF: no transitive dependency.
- BCNF: for every FD X -> Y, X is a super key (3NF may allow Y prime). BCNF may lose dependency preservation.
- Surrogate key: system-generated, meaningless, never-changing primary key.
- MVD X ->> Y: independent multi-valued facts. 4NF: BCNF + no non-trivial MVD.
- 5NF: no join dependency other than those implied by candidate keys.

Possible exam questions

1. What is normalization? Explain insertion, deletion and update anomalies with an example.
2. Define functional dependency. Explain Armstrong's axioms and attribute closure with an example.
3. Explain lossless-join and dependency-preserving decomposition.
4. Explain 1NF, 2NF and 3NF with examples.
5. Define BCNF. Differentiate 3NF and BCNF with an example.
6. What is a surrogate key? Compare with a natural key.
7. Explain multivalued dependency and 4NF with an example.
8. Explain join dependency and 5NF with an example.

DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT – V : Transactions, Concurrency, Recovery & Indexing

Easy handwritten notes with diagrams for students

Topics in this unit

1. Transaction concept, states, ACID properties
2. Concurrent executions, serializability, recoverability
3. Implementation of isolation, testing for serializability
4. Concurrency protocols: lock based, timestamp based, optimistic
5. Deadlocks
6. Failure classification, storage, recovery and atomicity, recovery algorithm
7. Indexing: B+ trees and operations
8. Hash based indexing

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for Serializability, lock based, time stamp based, optimistic, concurrency protocols, Deadlocks, Failure Classification, Storage, Recovery and Atomicity, Recovery algorithm. Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing.

Part I : Transaction Concept

1.1 What is a Transaction?

Definition: A **transaction** is a **logical unit of work**: a sequence of read / write operations on the database that must be executed **completely or not at all**. It starts with BEGIN and ends with COMMIT or ROLLBACK.

Example - transfer Rs. 500 from account A to account B:

```
T1:  read(A);       A = A - 500;   write(A);
     read(B);       B = B + 500;   write(B);
     commit;
```

If the system fails after **write(A)** but before **write(B)**, money disappears. The transaction concept prevents this.

Basic operations: **read(X)** copies X from the database to a local buffer; **write(X)** writes the local value back to the database; **commit** makes changes permanent; **rollback / abort** undoes all changes.

1.2 ACID Properties

Property	Meaning	Ensured by	Example
Atomicity	All operations happen or none (all-or-nothing)	Recovery manager (log, undo)	Debit and credit both done or both undone
Consistency	DB moves from one valid state to another valid state; constraints hold	Programmer + integrity constraints	Total A + B same before and after
Isolation	Concurrent transactions do not interfere; appears as if run one by one	Concurrency-control manager (locks)	T2 cannot see T1's half-done data
Durability	Once committed, changes survive any crash	Recovery manager (log on stable storage)	Committed transfer remains after power failure

1.3 Transaction States

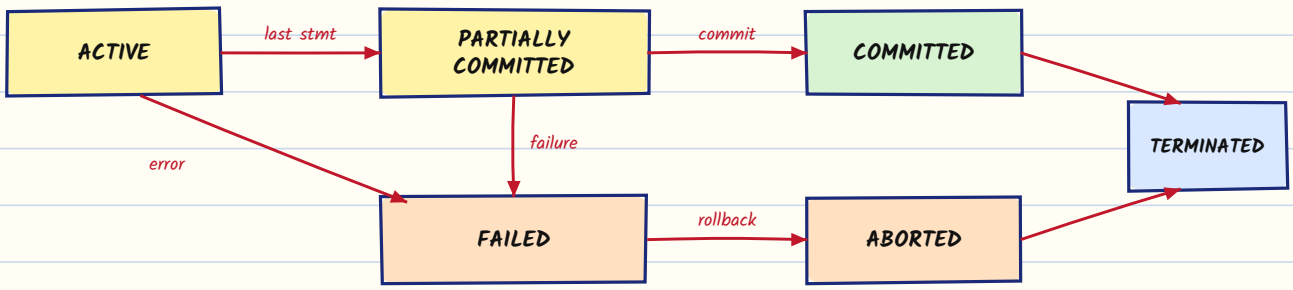


Fig 5.1 : Transaction state diagram

State	Meaning
Active	Initial state; transaction is executing its operations
Partially committed	After the final statement; results may still be only in buffer
Committed	Changes are safely written to the database - permanent

State	Meaning
Failed	Normal execution cannot continue (error / crash)
Aborted	Changes rolled back, DB restored to state before the transaction. Then restart (if no logical error) or kill
Terminated	Transaction has left the system

Part 2 : Concurrent Executions

Running many transactions at the same time (interleaving their operations) gives higher **throughput**, better **resource utilisation** and lower **waiting time**, but may produce wrong results if not controlled. A **schedule** is the order in which operations of transactions are executed.

2.1 Problems of Uncontrolled Concurrency

Problem	What happens	Example
Lost update (W-W)	Two transactions update the same item; one overwrites the other	T1 and T2 both read A=100; T1 writes 90, T2 writes 120 -> T1's update is lost
Dirty read (W-R)	T2 reads a value written by T1 that is not yet committed; T1 then aborts	T2 used data that never really existed
Unrepeatable read (R-W)	T1 reads X twice and gets different values because T2 changed X in between	Price shows 50 then 60 in same transaction
Phantom read	A repeated range query returns new rows inserted by another transaction	COUNT(*) is 10, later 11

2.2 Serial and Non-serial Schedules

- **Serial schedule** - transactions run one after another with no interleaving. Always correct (n! possible orders) but slow.
- **Non-serial schedule** - operations interleaved; fast but must be equivalent to some serial schedule to be correct.
- A non-serial schedule that is equivalent to a serial schedule is **serializable**.

2.3 Serializability

(a) Conflict serializability

Two operations **conflict** if they belong to different transactions, access the **same data item** and at least one is a **write**.

T1	T2	Conflict?
read(X)	read(X)	No
read(X)	write(X)	Yes (R-W)
write(X)	read(X)	Yes (W-R)
write(X)	write(X)	Yes (W-W)
any on X	any on Y (different item)	No

A schedule is **conflict serializable** if it can be converted to a serial schedule by swapping adjacent **non-conflicting** operations.

(b) View serializability

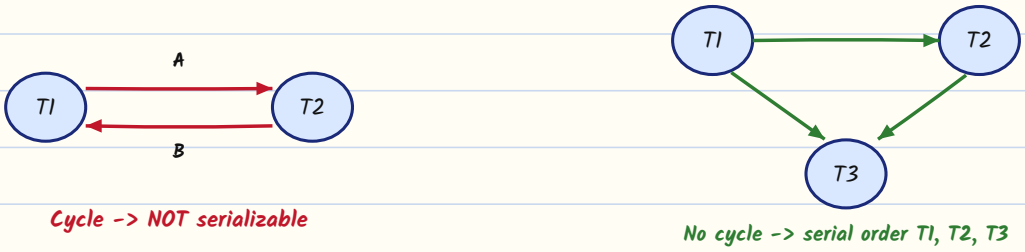
Schedules S and S' are **view equivalent** if: (1) each transaction reads the same initial value of an item in both, (2) each read gets its value from the same writer in both, (3) the final write of every item is by the same transaction. Every conflict-serializable schedule is view-serializable, but not the reverse (view serializable schedules containing **blind writes** may not be conflict serializable). Testing view serializability is NP-complete.

2.4 Testing for Serializability – Precedence Graph

Build a **precedence (serialization) graph**: one node per transaction; draw edge Ti -> Tj if an operation of Ti conflicts with a LATER operation of Tj. **If the graph has a cycle, the schedule is NOT conflict serializable; if it has no cycle, it is serializable**, and a topological order of the graph gives the equivalent serial order.

Schedule S1	T1	T2
1	read(A); write(A)	
2		read(A); write(A)
3		read(B); write(B)
4	read(B); write(B)	

Item A: T1 writes before T2 touches it -> edge T1 -> T2. Item B: T2 writes before T1 touches it -> edge T2 -> T1. Result: a **cycle**.



Cycle -> NOT serializable

No cycle -> serial order T1, T2, T3

Fig S.2 : Precedence graphs – left has a cycle, right has none

2.5 Recoverability

A schedule must allow the system to recover correctly from failures. The key question: if T2 reads data written by T1, what if T1 aborts?

Schedule type	Rule	Property
Recoverable	If Tj reads data written by Ti, then Ti must commit **before** Tj commits	Avoids committed transaction having read uncommitted-then-aborted data
Cascadeless (ACA)	Tj may read an item only after the writer Ti has **committed**	No cascading rollback (one abort does not force others to abort)
Strict	No transaction reads OR overwrites an item written by Ti until Ti commits / aborts	Easy undo; used by most DBMS

Relationship: **Strict** ⊂ **Cascadeless** ⊂ **Recoverable**. A **cascading rollback** happens when abort of Ti forces abort of Tj that read Ti's dirty data, and so on.

2.6 Implementation of Isolation (Isolation levels)

Full isolation (serializability) reduces concurrency, so SQL defines weaker levels. A higher level prevents more anomalies but allows less concurrency.

Isolation level	Dirty read	Unrepeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Isolation is implemented using **locking** (2PL), **timestamps**, **optimistic validation** or **multiversion (snapshot) isolation**, explained next.

Part 3 : Concurrency Control Protocols

3.1 Lock-Based Protocols

A **lock** is a variable attached to a data item that controls access to it. A transaction must obtain a lock before using the item.

Lock mode	Meaning	Operation
Shared (S)	Many transactions can hold it together	Read only: lock-S(X)
Exclusive (X)	Only one transaction can hold it	Read and write: lock-X(X)

Requested \ Held	S	X
S	Compatible (YES)	NO
X	NO	NO

Lock compatibility matrix

Release with unlock(X). Using locks alone is not enough: if T locks and unlocks carelessly, non-serializable results still occur. So we need a **locking protocol**.

Two-Phase Locking (2PL)

Each transaction acquires and releases locks in two phases:

- **Growing phase** - can obtain locks, cannot release any.
- **Shrinking phase** - can release locks, cannot obtain any new lock.
- The point where the last lock is obtained is the **lock point**. Transactions can be serialized in order of their lock points.

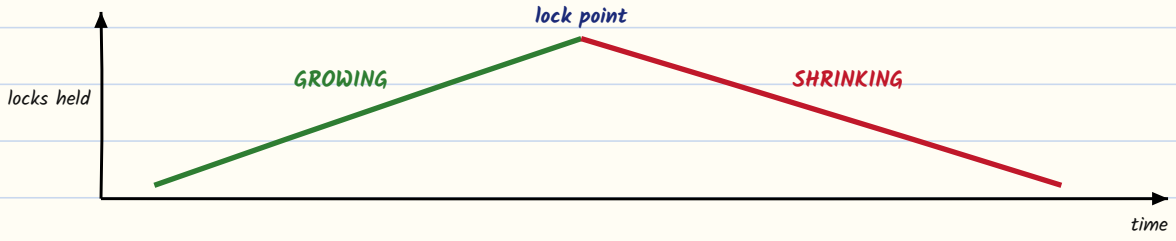


Fig 5.3 : Two-phase locking

Variant	Rule	Benefit
Basic 2PL	Growing then shrinking	Guarantees conflict serializability; may cause deadlock and cascading rollback

Variant	Rule	Benefit
Strict 2PL	Hold all **exclusive** locks until commit / abort	Cascadeless schedules
Rigorous 2PL	Hold **all** locks (S and X) until commit / abort	Serial order = commit order
Conservative 2PL	Lock all items before starting	Deadlock-free but hard to know items in advance

Lock conversion: upgrade S -> X in growing phase; downgrade X -> S in shrinking phase. Problems of locking: **deadlock** and **starvation**.

3.2 Timestamp-Based Protocols

Each transaction T gets a unique **timestamp TS(T)** when it starts (older = smaller). Each item Q keeps **R-TS(Q)** (largest TS that read Q) and **W-TS(Q)** (largest TS that wrote Q). The protocol makes the schedule equivalent to the serial order of timestamps. No locks, so **no deadlock**.

Operation by T on Q	Condition	Action
read(Q)	TS(T) < W-TS(Q) (Q already overwritten by a younger T)	Reject read, **rollback T** and restart with new timestamp
	TS(T) ≥ W-TS(Q)	Execute; R-TS(Q) = max(R-TS(Q), TS(T))
write(Q)	TS(T) < R-TS(Q) (a younger T already read Q)	Reject write, rollback T
	TS(T) < W-TS(Q) (a younger T already wrote Q)	Reject write, rollback T (Thomas' write rule: just **ignore** this obsolete write)
	Otherwise	Execute; W-TS(Q) = TS(T)

- Ensures conflict serializability and freedom from deadlock.
- Drawbacks: possible **starvation** of long transactions; schedules may not be recoverable (fix by delaying commits).

3.3 Optimistic (Validation-Based) Protocol

Assumes conflicts are **rare**, so transactions run without locks and are checked only at the end. Good for read-mostly workloads.

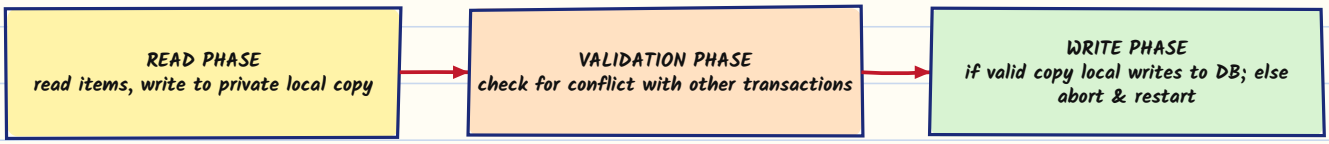


Fig 5.4 : Three phases of optimistic concurrency control

Validation test for Tj against every earlier transaction Ti (TS(Ti) < TS(Tj)): Tj passes if either (1) Ti finishes before Tj starts, or (2) Ti's write set does not intersect Tj's read set and Ti finishes its write phase before Tj starts its write phase. Otherwise Tj is **rolled back**.

* **Compare:** Locking = **pessimistic** (prevent conflicts, may wait / deadlock). Timestamp = order by age. Optimistic = **allow, then validate**, aborts on conflict. Multiversion (MVCC) keeps old versions so readers never block writers.

Part 4 : Deadlocks

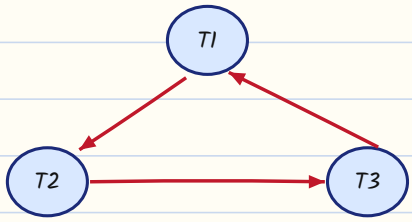
Definition: A **deadlock** is a situation in which two or more transactions are each waiting for a lock held by another, so **none can proceed**.

Example: T1 holds lock on A and requests B; T2 holds lock on B and requests A.

Four necessary (Coffman) conditions: **mutual exclusion**, **hold and wait**, **no preemption**, **circular wait**.

4.1 Deadlock Detection - Wait-for Graph

Node per transaction; edge Ti -> Tj if Ti is waiting for a lock held by Tj. **A cycle means deadlock**. The system checks the graph periodically.



Cycle T1 -> T2 -> T3 -> T1
= DEADLOCK

Fig 5.5 : Wait-for graph with a cycle

4.2 Deadlock Prevention

Scheme	If Ti requests a lock held by Tj	Type
Wait-Die	If Ti is **older** (smaller TS) it waits; if Ti is younger it **dies** (rolled back, restarts with same TS)	Non-preemptive
Wound-Wait	If Ti is **older** it **wounds** (aborts) Tj; if Ti is younger it waits	Preemptive
Timeout	Wait for fixed time, then roll back	Simple, may roll back needlessly
Conservative 2PL	Acquire all locks at the start	No hold-and-wait

4.3 Deadlock Recovery

- **Victim selection** - choose transaction with least cost (shortest running, fewest locks, least work done).
- **Rollback** - total (abort victim) or partial (back to a point that breaks the deadlock).
- Prevent **starvation** - include the number of rollbacks in the cost so the same victim is not always chosen.

Part 5 : Failures, Storage and Recovery

5.1 Failure Classification

Failure	Cause	Effect / recovery
Transaction failure	Logical error (bad input, overflow) or system error (deadlock)	Only that transaction aborted; rollback
System crash	Power cut, OS / hardware / software bug	Contents of main memory lost; disk safe. Restart recovery needed
Disk failure	Head crash, bad sectors, disk damage	Data on disk lost; recover from backup / archive + log

5.2 Storage Types

Storage	Survives system crash?	Examples
Volatile	No	Main memory, cache
Non-volatile	Yes (but disk can fail)	Disk, SSD, tape
Stable	Yes - never lost (theoretical); implemented by **replicating** data on several non-volatile devices	Mirrored disks, remote backup

Recovery must ensure atomicity and durability: updates of uncommitted transactions must be **undone** and updates of committed transactions must be **redone** if lost.

5.3 Log-Based Recovery (Atomicity and Durability)

The **log** is a sequence of records kept on stable storage. **Write-Ahead Logging (WAL)**: the log record must be written to stable storage **before** the corresponding data page is written to disk.

Log record	Meaning
<Ti, start>	Transaction Ti has started
<Ti, X, V1, V2>	Ti changed item X from old value V1 to new value V2
<Ti, commit>	Ti committed
<Ti, abort>	Ti aborted
<checkpoint L>	Checkpoint; L = list of active transactions

	Deferred database modification	Immediate database modification
Write to DB	Only after commit	Any time, even before commit
Log needs	New value only (redo)	Old and new value (undo + redo)
On crash	REDO committed transactions; ignore uncommitted	UNDO uncommitted, REDO committed

Recovery rule: after a crash scan the log; if it has both <Ti, start> and <Ti, commit> -> **redo(Ti)**; if it has <Ti, start> but no commit -> **undo(Ti)**.

Checkpoints

Scanning the whole log is slow. A **checkpoint** periodically forces all log records and modified buffer pages to disk and writes a checkpoint record. After a crash, only the transactions active at or after the last checkpoint need to be examined.

Shadow paging (alternative)

Keeps two page tables: **current** and **shadow**. Updates go to new pages via the current table; the shadow table is untouched. Commit = atomically make the current table the new shadow. Abort = discard the current table. No log is needed, but data gets fragmented and garbage must be collected.

5.4 Recovery Algorithm (ARIES)

ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) is the standard algorithm. It uses WAL, **repeating history during redo** and logging of undo (CLR - compensation log records). Each log record has a **LSN** (log sequence number); each page stores PageLSN. Two tables are kept: **Dirty Page Table** and **Transaction Table**.

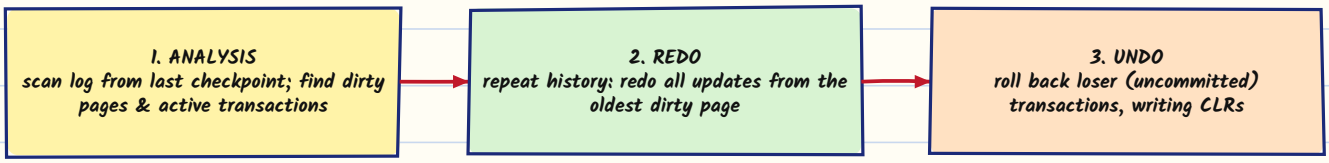


Fig 5.6 : Three passes of ARIES recovery

Pass	Work
Analysis	Starts at last checkpoint; rebuilds Dirty Page Table and Transaction Table; finds RedoLSN (where redo starts) and the loser transactions
Redo	Scans forward from RedoLSN; reapplies every logged update (committed or not) if page's PageLSN < record's LSN
Undo	Scans backward undoing updates of loser transactions; writes a CLR for each undo so work is never undone twice

Part 6 : Indexing Techniques

An **index** is an auxiliary data structure (like the index of a book) that lets the DBMS find rows by a **search key** without scanning the whole table. Indexes speed up reads but slow down inserts / updates and take space.

Index type	Meaning
Primary (clustering)	Index on the attribute by which the file is physically sorted
Secondary (non-clustering)	Index on another attribute; file is not sorted by it
Dense	One index entry for every record
Sparse	Index entry only for some records (one per block)
Ordered vs hash	Ordered: tree / sorted (B+ tree) - good for range queries. Hash: hash function - good for equality queries

6.1 B+ Tree

The **B+ tree** is a balanced multi-way search tree and the most common index structure (used in almost every RDBMS). All leaves are at the **same depth**, so every search costs the same: O(log n) disk accesses.

Properties (order n = maximum number of pointers in a node)

Node	Keys	Pointers / children
Root	at least 1 key	at least 2 children (unless it is a leaf)
Internal node	$\lceil (n-1)/2 \rceil$ to n-1	$\lceil n/2 \rceil$ to n children

Node	Keys	Pointers / children
Leaf node	$\lceil (n-1)/2 \rceil$ to n-1 keys + data pointers	last pointer goes to the next leaf

- Internal nodes store only **keys as guides**; **all actual data pointers are in the leaves**.
- Leaves are linked as a **sorted linked list** -> fast range queries and sequential scans.
- Difference from B tree: B tree stores data pointers in internal nodes too; B+ tree has more keys per internal node (shallower), faster range scan.

Example B+ tree of order 4 (max 3 keys, min 2 keys per leaf):

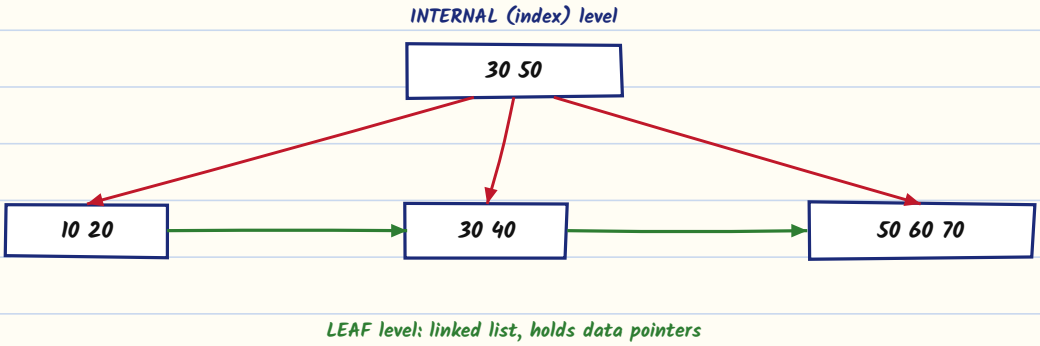


Fig 5.7 : B+ tree (order 4)

6.2 Operations on B+ Trees

Search

Start at the root; at each node follow the pointer for the key range containing the search key; at the leaf look for the key. For a range query, find the first key then follow the leaf links.

Insertion

- Find the correct leaf and insert the key in sorted order.
- If the leaf has room -> done.
- If the leaf **overflows** -> split it into two leaves; the **smallest key of the new right leaf is copied up** to the parent (leaf split: copy-up).
- If the parent overflows -> split it; the **middle key is moved up** (internal split: push-up). Splits may reach the root and increase tree height by one.

Example: insert 80 into the tree above. Leaf [50 60 70] becomes [50 60 70 80] (4 keys, overflow) -> split into [50 60] and [70 80]; key 70 is copied to the root:

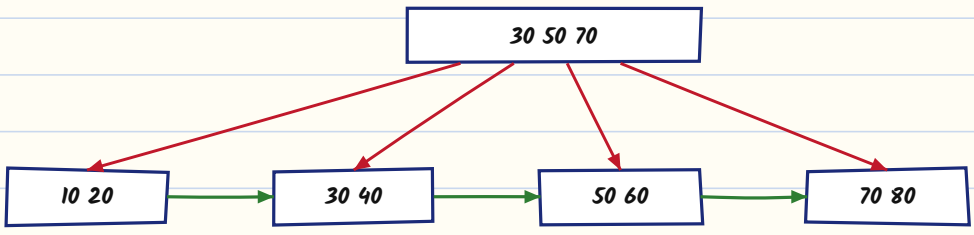


Fig 5.8 : B+ tree after inserting 80 (leaf split, 70 copied up)

Deletion

- Find the leaf and delete the key.

- If the leaf still has the minimum number of keys -> done (update parent key if the smallest key was removed).
- If it **underflows** -> first try to **borrow (redistribute)** a key from a sibling that has more than the minimum; update the parent separator.
- If the sibling cannot spare a key -> **merge** with the sibling and delete the separator key from the parent. Merges can propagate upward and may reduce the height.

Operation	Cost (disk accesses)
Search / insert / delete	$O(\log_f N)$, where f = fan-out (typically 100-500) -> 3 or 4 levels for millions of rows
Range query	Search first key + sequential leaf scan

6.3 Hash-Based Indexing

A **hash function** h(K) maps a search key directly to a **bucket** (a disk page holding records). Very fast for **equality search** (about 1 access) but **cannot do range queries** because order is lost.

Static hashing

- Number of buckets B is fixed. Bucket = h(K) = K mod B.
- Bucket full -> **overflow chain** of extra pages. Many overflows make search slow; too many empty buckets waste space; changing B needs full rehash.

Extendible hashing (dynamic)

Uses a **directory** of 2^d pointers (d = **global depth**) indexed by the first d bits of the hash value. Each bucket has a **local depth** d'. When a bucket is full it is **split**; if d' = d the directory is **doubled**. No overflow chains and no full rehash.

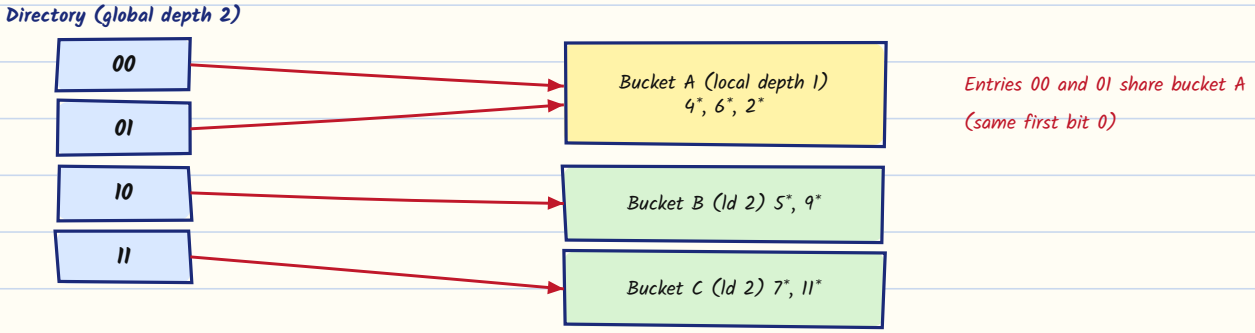


Fig 5.9 : Extendible hashing directory and buckets

Linear hashing

Grows the number of buckets **one at a time** in a fixed order using a **split pointer**, so **no directory** is needed. When a split condition occurs (overflow or load factor exceeded) the bucket at the split pointer is split (not necessarily the overflowing one) using the next hash function $h_{level+1}$; overflow chains are used temporarily.

	Static	Extendible	Linear
Number of buckets	Fixed	Grows by splitting; directory doubles	Grows one bucket at a time
Directory	No	Yes	No
Overflow pages	Yes, many	No (almost)	Yes, temporary
Best for	Stable data size	Fast growth, uniform	Smooth growth

	B+ tree index	Hash index
Equality search	Good, O(log N)	Best, about O(1)
Range search	Excellent (linked leaves)	Not supported
Sorted order output	Yes	No
Used for	Most indexes, primary keys, ORDER BY	Exact-match lookups

```
CREATE INDEX idx_name ON STUDENT (Name);              -- B+ tree (default in most DBMS)
CREATE UNIQUE INDEX idx_email ON STUDENT (Email);
CREATE INDEX idx_dept_age ON STUDENT (Dept_id, Age);   -- composite index
DROP INDEX idx_name;                                   -- (MySQL: DROP INDEX idx_name ON STUDENT;)
```

Quick Revision - Unit V

- Transaction = logical unit of work. ACID: Atomicity, Consistency, Isolation, Durability.
- States: active -> partially committed -> committed; active/partially committed -> failed -> aborted -> terminated.
- Concurrency problems: lost update, dirty read, unrepeatable read, phantom.
- Serializable = equivalent to a serial schedule. Conflict test = precedence graph (cycle => not serializable).
- Recoverable, cascadeless, strict schedules. Isolation levels: read uncommitted ... serializable.
- Locks: S and X; 2PL (growing + shrinking), strict 2PL, rigorous 2PL.
- Timestamp ordering: R-TS, W-TS, rollback when order violated; Thomas write rule. Optimistic: read, validate, write.
- Deadlock: wait-for graph cycle. Prevention: wait-die, wound-wait. Recovery: victim selection, rollback.
- Failures: transaction, system crash, disk. Storage: volatile, non-volatile, stable.
- Log-based recovery, WAL, deferred / immediate update, checkpoint, shadow paging. ARIES: analysis, redo, undo.
- B+ tree: balanced, data in linked leaves, split (copy-up) on insert, borrow / merge on delete.
- Hashing: static, extendible (directory, global / local depth), linear (split pointer); no range queries.

Possible exam questions

1. Explain ACID properties and transaction states with a diagram.
2. Explain the problems caused by concurrent execution with examples.
3. What is serializability? Explain conflict and view serializability and test it using a precedence graph.
4. Explain recoverable, cascadeless and strict schedules; isolation levels.
5. Explain the two-phase locking protocol and its variants.
6. Explain timestamp-based and optimistic (validation) protocols.
7. What is deadlock? Explain detection, prevention (wait-die / wound-wait) and recovery.
8. Explain failure classification, log-based recovery, checkpoints and the ARIES algorithm.
9. Explain B+ tree structure and the insertion / deletion operations with examples.
10. Explain static, extendible and linear hashing.